

Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی Claude Certified Architect — Foundations Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) — یعنی بنیادی علم کا جائزہ لیتا ہے — کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں۔ Claude و بنیادی ٹیکنالوجیز جن سے

agentic systems امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines، ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity، structured data بنانا، اور غیر ساختہ دستاویزات سے tools نکالنا۔

هدف امیدوار

کرتا ہے آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- Context windows** — لمبی دستاویزات کے ساتھ کام، multi-agent context passing
- CI/CD pipelines** — automated code review، test generation
- Escalation and reliability** — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (میں سے 1 درست 4)
اسکورنگ	100–1000 scale، passing score 720
Guessing penalty	نہیں (سوال کا جواب دیں!)
Scenarios	8 4 (randomly selected) میں سے

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	27%
2. Tool design and MCP integration	18%
3. Claude Code configuration and workflows	20%
4. Prompt engineering and structured output	20%
5. Context management and reliability	15%

امتحانی منظر نامہ

Scenario 1: Customer Support Agent

returns, billing disputes, اور account issues استعمال کرتے ہوئے Claude Agent SDK بناتے ہیں جو آپ ایک agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) سنبھالتا ہے۔ مناسب first-contact resolution +% استعمال کرنا 80 دفعہ (`escalate_to_human`) ساتھ ساتھ

Scenario 2: Claude Code ساتھ Code Generation

code generation, refactoring, تیز کرنے کے لیے استعمال کرتے ہیں development کو Claude Code آپ کے ساتھ CLAUDE.md configuration اور custom slash commands آپ کو اسدے debugging, documentation کے ساتھ integrate کب استعمال کرنا planning mode کرنا، اور یہ سمجھنا کے

Scenario 3: Multi-Agent Research System

tasks کو coordinator agent specialized subagents سونپتا: web research, document analysis, synthesis, تیار کرنے چاہئیں reports کے ساتھ مکمل citations سسٹم کو report generation اور

Scenario 4: Developer Productivity Tools

agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) میں مدد دیتا ہے۔ automate کیے جاتے ہیں

Scenario 5: Claude Code for Continuous Integration

pull اور automated code reviews, test generation, Claude Code کو CI/CD pipeline تاکہ کم سے کم false positives ڈیزائن ہونے چاہئیں کے Prompts حاصل ہو۔ request feedback

Scenario 6: Structured Data Extraction

کرتا ہے، validate کے ذریعے JSON schemas کو output، سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے اور high accuracy کے لیے edge cases پر سنبھالنا چاہیے۔ اس کا برقرار رکھنا اس کے لیے ضروری ہے۔

Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں جن میں context window management، turns متعدد، memory strategies، tool design کے لیے execution محفوظ، کا برقرار رکھنا instructions میں اور مہم یا متضاد، user inputs کو سنبھالنا شامل ہے۔

Scenario 8: Agentic AI Tools (ہماری مدد کریں — مواد موجود نہیں)

میں شامل نہیں ہے۔ guide کے لیے ابھی تک اس candidates کی اطلاع امتحان دینے والے scenario اس میں share میں GitHub Issues کے سوالات دیکھیں، تو ان میں scenario کے اصل امتحان میں اس شامل کر سکیں۔ آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کے لیے coverage تاکہ مہم مکمل رہے۔

سرکاری دستاویزات

وسیلہ	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills

Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

نظریاتی بنیادیں: حصہ ۱

یہ حصہ تمام نظریاتی احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو کے مطابق — اس سے آپ کے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts ٹیکنالوجیز اور موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں۔

کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: [Messages API](#) | [Prompt Engineering](#)

1.1 API Request Structure

Claude API کی request کی ر Claude Messages API کی پیروی کرتا ہے request-response model ایک شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"},
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

□□ م فیلڈر:

- `model` — model selection (`claude-opus-4-6` , `claude-sonnet-4-6` , `claude-haiku-4-5`)
- `max_tokens` — response □□ زیاد □□ زیاد کی تعداد tokens میں زیاد □□ زیاد
- `system` — system prompt (model □□ کی تعریف کرتا □□)
- `messages` — conversation history (history تسلسل برقرار رکھنے □□ لیں □□ آپ کو مکمل) □□)
- `tools` — tools کی definitions دستیاب
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array استعمال کرتا □□ instructional role اور ایک conversational roles دو:

- `user` — user messages, `tool_result` □□ اندر `user`-role message (ایک □□ `tool_result` content block □□ کی صورت میں بھیج □□ جاتا □□، ایک الگ `tool` role □□ طور پر □□)
- `assistant` — model responses (history □□ شامل کی □□ جاتا □□، (بھیجت □□ وقت شامل کی □□ جاتا □□، `tool_use` content blocks)
- `system` — top-level `system` field □□ سیٹ کیا جا سکتا □□ (□□ turn □□ □□) یا (س □□ لاگو □□ وٹا □□ □□ □□) □□ ذریع □□ سیٹ کیا جا سکتا □□ اس نقط □□ س □□ آگ □□ شامل کیا جا سکتا □□ inline □□ طور پر `{"role": "system", ...}` میں `messages` □□ تاج □□ دیکھیں placement rules لاگو □□ وٹا □□، مخصوص

□□ طور `user`-role message □□ طور پر □□ میں بھیجا جاتا □□ ایک message □□ role `"tool"` کو Tool results □□ شامل □□ وٹا □□ `tool_result` content block □□ میں ایک content پر بھیج □□ جاتا □□ میں جس □□

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01...",
      "content": "...
    }
  ]
}
```

top-level کے طور پر ظاہر ہو سکتا ہے، نہ صرف role میں بھی ایک array messages براہ راست system کے بغیر invalidate prefix کو cached field کے top-level کے ذریعے اس کا مقصد system parameter placement rules شامل کرنا ہے اس کے مخصوص instructions کے درمیان conversation:

- پر ختم ہونے والے server tool use یا (ہوں blocks tool_result بشمول وہ جس میں) user turn یا ایک کے فوراً بعد آنا چاہیے assistant turn
- ہونا چاہیے element کا آخری array سد پہلے آنا چاہیے یا assistant turn کسی
- ملتا error کے درمیان نہیں آ سکتا — ایسا کرنے پر 400 tool_result اور اس کے block tool_use کسی
- پہلے والوں پر اور بعد (کے درمیان شامل کیے گئے conversation بشمول) system messages بعد میں آنے والے پر فوقیت رکھتے ہیں field کے top-level کے لیے turns کے

Model requests بھیجی جا رہی ہیں conversation history میں آپ کو مکمل API request انٹیلیجنس: م۔ ہر آزاد ہوتا ہے call محفوظ نہیں رکھتا — ہر state درمیان

1.3 Response میں stop_reason Field

کیوں روکی generation کے model شامل ہوتا ہے، جو بتاتا ہے کے stop_reason میں Claude API response:

Value	Description	Action
"end_turn"	مکمل کر لیا response کے Model	کو دکھائیں user نتیجہ
"tool_use"	کرنا چاہتا ہے Model tool call	واپس دیں result کریں اور Tool execute
"max_tokens"	Token limit ختم ہو گئی	Response truncated ہے ممکن ہے limit ہے بڑھانی پڑے
"stop_sequence"	Stop sequence مل گئی	کریں handle کے مطابق application logic اپنی

Agentic systems میں "end_turn" اور "tool_use" — یہی agent loop سب سے اہم ہیں —

1.4 System Prompt

متعین کرنا ہے behavioral rules اور context کے جو instruction ایک خاص System prompt:

- میں دیا جاتا ہے field system کا حصہ نہیں ہوتا؛ الگ array messages

- user messages پر فوقیت رکھتا ہے
- میں لاگو رہتا ہے conversation ہوتا ہے اور پوری load ایک بار
- role, constraints, اور output format استعمال ہوتا ہے

پیدا کر سکتی ہے مثال کے طور پر tool associations کو غیر ارادی wording کی system prompt میں لے کر ہم کو ضرورت سے زیادہ `get_customer` کو model کی تصدیق کریں "جیسی کہ ادیت customer پر،" ہمیشہ استعمال کرنے پر مجبور کر سکتی ہے، حتیٰ کہ جب یہ ضروری نہ ہو۔

1.5 Context Window

Context window text (tokens) □□ جس model کی کل مقدار □□ اس میں process ایک وقت میں model کے □□ کر سکتا □□

□□ شامل:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

مسائل context-window

1. **Last-in-the-middle effect:** models input لمبے اعتماد سہ کو زیادہ سے زیادہ پر موجود معلومات کو زیادہ سے زیادہ استعمال کرتے ہیں۔ اگرچہ یہ ایک مشکل ہے، مگر درمیان کی تفصیلات process کر سکتے ہیں۔ حل: اہم معلومات شروع یا آخر کے قریب miss کرتے ہیں، مگر درمیان کی تفصیلات رکھیں
2. **Tool results** واپس کر کے tool 40+ fields شامل کرتا ہے۔ اگر output میں tool call context کا جمع ہونا: ہر tool کا بڑا حصہ ضائع ہو جاتا ہے۔ context میں صرف 5 ام ویوں، تو
3. **Progressive summarization:** history compress کرتے وقت numeric values, percentages, dates اور گیم وغیرہ واضح ہو جاتے ہیں ("تقریباً"، "کچھ"، "چند")

tool_use اور Tools: باب 2

Tool Use: دستاویزات

2.1 tool_use ☐ کیا

برای Model کی اجازت دینا call external functions کو Claude جو mechanism ایک `tool_use` result کرتا ہے اور execute اس code بنانا ہے؛ آپ کا structured tool call request میں چلانا — وہ code راست واپس کرتا ہے

2.2 Tool Definition

کے: کیا جاتا ہے define کے ذریعے JSON schema کی tool

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

کے: انتہائی اہم پہلو Tool description

- Description tool selection mechanism** LLM tools کی بنیاد پر descriptions کو ان کی Minimal descriptions ("Customer information retrieve کرتا کرتا" اُن مواقع پر غلطیوں کا "کرتا کرتا" overlap ایک دوسرے سے tools باعث بنتی ہیں جب کرتے ہیں
- Description میں شامل کریں:**
 - Tool اور کیا واپس کرتا کرتا
 - Input formats اور example values
 - Edge cases اور constraints
 - مقابلہ میں کب استعمال ہو tool similar alternatives
- analyze_content اور analyze_document** سہجے ہیں اگر **overlapping descriptions** ایک جیسے یا انہیں خلط ملط کر دے گا model تقریباً ایک جیسی ہوں، تو descriptions
- Built-in tools MCP tools:** agents similar functionality والے built-in tools (Read, Grep) MCP tools — مضبوط بنائیں MCP tool descriptions پر ترجیح دے سکتے ہیں اس سے بچدے کے لیے concrete advantages, unique data, context یا و فرام نہیں کر سکتے ہیں

2.3 tool_choice Parameter

کیسے منتخب کرتا کرتا model tools کنٹرول کرتا کرتا tool_choice

Value	Behavior	کب استعمال کریں
<code>{ "type": "auto" }</code>	Model کرنا tool call خود طے کرتا کرتا کرنا text یا میں جواب دینا	default زیادہ تر حالات میں
<code>{ "type": "any" }</code>	کرنا ہوگی tool call کو لازماً کوئی Model	guaranteed جب آپ کو structured output چاہیں
<code>{ "type": "tool", "name": "extract_metadata" }</code>	کرنی tool call کو لازماً ایک مخصوص ہوگی	forced first step / execution order جب آپ کو چاہیں

2. **Nullable fields:** ایسی معلومات کے لیے `"type": ["string", "null"]` طور پر استعمال کریں جو ممکنہ طور پر `null` ہو سکتا ہے۔
3. **Enums with "other":** predefined categories کے لیے `"other"` شامل کریں + ایک detail string
4. **Enum "unclear":** جب `"unclear"` category غلط ہو تو — ایماندار model category اُن صورتوں کے لیے جب `"unclear"` category غلط ہو

2.5 Syntax Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON, field type غلط	JSON schema کے ساتھ <code>tool_use</code> (ختم کر دیتا)
Semantic	Totals match نہیں کرتے، value field غلط، hallucination	Validation checks, feedback کے ساتھ retry, self-correction

بنانا Claude Agent SDK — Agentic Systems: باب 3

دستاویزات: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

3.1 Agentic Loop کیا ہے

Agentic loop کی ایک actions صرف جواب نہیں دیتا — وہ Model کے pattern کا بنیادی task execution خودکار sequence انجام دیتا ہے:

- Claude کو tools کے ساتھ request بھیجیں
- Response وصول کریں
- stop_reason چیک کریں:
 - "tool_use" -> tool execute کریں، result history میں append کریں step 1 پر واپس جائیں
 - "end_turn" -> task مکمل، result user کو دکھائیں
- کریں repeat مکمل ہونے تک

model-driven approach: Claude اپنے context اور tool results پر اگلا tool منتخب کی بنیاد پر۔
تو fixed ہوتا ہے۔ action sequence کے مختلف جہاں hard-coded decision trees کرتا ہے۔

Anti-patterns (ان سے بچیں):

- Completion detect کرنا ("Task completed")
- Primary stop condition طور پر arbitrary iteration limit (مثلاً `max_iterations=5`)
- سمجھنا completion signal دیا یا نہیں، اس کے textual content assistant کے چیک کرنا

درست طریقہ: completion signal کا واحد قابل اعتماد `stop_reason == "end_turn"`

3.2 AgentDefinition Configuration

AgentDefinition Claude Agent SDK میں agent configuration object ہے:

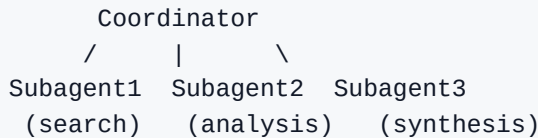
```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # Coordinator کے لیے:
    # allowed_tools=["Task", "get_customer", ...]
)
```

اس کے parameters:

- `name` / `description` — agent کی شناخت اور وضاحت
- `system_prompt` — system prompt کی ہدایات والا
- `allowed_tools` — (least privilege اصول) کی فہرست tools اجازت یافتہ

3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture میں بنائی جاتی ہے hub-and-spoke topology عموماً



اس کے ذمہ دار ہیں Coordinator:

- Task کو subtasks میں تقسیم کرنا
- (dynamic selection) درکار ہیں subagents کو طے کرنا کہ کون سا
- کو سونپنا subagents کام
- کرنا validate نتائج جمع اور
- Errors اور retries handle کرنا
- تک پہنچانا user نتائج

الگ الگ ہوتا ہے context کا subagents: اس کے اصول

- Subagents کو conversation history کی coordinator خود بخود inherit نہیں کرتے
- میں واضح طور پر دینا ہوتا ہے context subagent prompt تمام ضروری
- Subagents calls memory share کے درمیان
- (observability اور error control کے ذریعے گزرتی ہے communication coordinator تمام کے لیے)

3.4 Subagents **Task Tool** بنانے کے لیے

Subagents **Task tool** کے ذریعے spawn کیے جاتے ہیں:

```
# Coordinator کے allowedTools میں "Task" شامل ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing لازمی ہے:

```
# Bad: subagent کے پاس context نہیں ہے
Task: "Analyze the document"

# Good: مکمل context prompt میں
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

Parallel spawning: subagents parallel — کر سکتا ہے Task s call میں متعدد response ایک coordinator چلتے ہیں:

```
# Coordinator کے response میں شامل ہونے والے response:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

3.5 Agent SDK میں Hooks

Hooks agent lifecycle کی اجازت دیتے ہیں transformation اور interception پر points کے مخصوص

PostToolUse tool result کو model پر لے کر intercept تک پہنچانے کے لیے SDK پر لے کر:

```
# MCP tools کے تاریخ کو formats normalize کریں مثال: مختلف
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy کو block کرنے کے لیے والے actions کی خلاف ورزی کرنے والے:

```
# کریں refunds block مثال: 500$ سے اوپر
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

Attribute	Hooks	Prompt instructions
Guarantee	Deterministic (100%)	Probabilistic (>90%, 100% نہیں)
کب استعمال کریں	Critical business rules, financial operations, compliance	General preferences, recommendations, formatting
مثال	Refunds > \$500 block کریں	"Try to solve before escalating"

استعمال کریں hooks، نہیں prompts — مالی، قانونی، یا حفاظتی نتائج کو failure قاعدہ: جب

باب 4: Model Context Protocol (MCP)

[Servers](#) | [Resources](#) | [Tools](#) | [MCP](#): دستاویزات

4.1 MCP کیا ہے

MCP سے جوڑنا Claude کو external systems جو open protocol ایک (MCP) Model Context Protocol:

- Tools** — functions agent actions میں لے کر انجام دینے کے لیے (CRUD operations, API calls, command execution)
- Resources** — context agent جسے data کے لیے (documentation, database schemas, content catalogs)
- Prompts** — predefined prompt templates کے لیے tasks عام

4.2 MCP Servers

فرام کرنا جب tools/resources کرتا ہے اور MCP protocol implement جو MCP server ایک process:

- جو جائے ہیں discover خود بخود تمام tools
- ایک ساتھ دستیاب ہوتے ہیں tools کے connected servers تمام
- انہیں کیسے استعمال کرے گا model طے کرتی ہیں کے Tool descriptions

4.3 MCP Servers کو Configure کرنا

Project configuration (`.mcp.json`) — team usage کے لیے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

اہم نکات:

- میں `.mcp.json` project root میں محفوظ ہوتا ہے اور version control میں ہوتا ہے
- Environment variables (`${GITHUB_TOKEN}`) secrets کے لیے استعمال ہوتے ہیں
- commit خود tokens — کے لیے استعمال ہوتے ہیں
- contributors کے لیے دستیاب ہوتا ہے

User configuration (`~/.claude.json`) — personal/experimental servers کے لیے:

- User کے home directory میں محفوظ ہوتا ہے
- Version control کے ذریعے share ہوتا ہے
- Personal experiments اور testing کے لیے

Servers کا انتخاب:

- Standard integrations (Jira, GitHub, Slack) موجود ہیں
- community MCP servers کے لیے پبلک سرور موجود ہیں
- کو ترجیح community MCP servers کے لیے پبلک سرور موجود ہیں
- کے لیے بنائیں unique, team-specific workflows صرف custom servers اپنی

4.4 MCP میں isError Flag

کو بتاتا ہے کہ agent استعمال ہوتا ہے یا نہیں `isError: true` میں response آتا ہے تو error میں MCP tool جب call fail ہوئی ہے

Structured error (اچھا):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

Generic error (anti-pattern):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

escalate بدلیں، یا query، کریں retry کو فیصلہ سازی کے لیے کوئی معلومات نہیں دیتا — کیا generic error agent کریں؟

4.5 MCP Resources

کیا: بغیر طلب کر سکتا actions حاصل کرنے کے لیے agent context میں جو ایسا Resources

- Content catalogs (hierarchical navigation، کی فہرست project tasks مثلاً تمام)
- Database schemas (data structure کے لیے سمجھنے کے لیے)
- Documentation (API references، internal guides)
- Issue/task summaries

Resource کا فائدہ: agent کے لیے exploratory tool calls کی ضرورت نہیں پڑتی کے لیے سمجھنے کے لیے agent کے لیے کا فائدہ Resource کا فائدہ: agent کے لیے exploratory tool calls کی ضرورت نہیں پڑتی کے لیے سمجھنے کے لیے

5 اور Claude Code — Configuration Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

5.1 CLAUDE.md Hierarchy

hierarchy استعمال کرتا ہے اس کی تین سطحی Claude Code میں جو instruction file(s) و CLAUDE.md ہے:

1. User-level: ~/.claude/CLAUDE.md
 - ہر لاگو user صرف اسی
 - نسخہ ہوتا ہے کہ share ذریعہ VCS
 - working style اور preferences ذاتی
2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
 - ہر لاگو project contributors تمام
 - ہوتا ہے کہ manage ذریعہ VCS
 - Coding standards, testing standards, architectural decisions
3. Directory-level: subdirectories میں CLAUDE.md
 - کہ ساتھ کام ہو تو لاگو ہوتا ہے کہ files کی directory جب اسی
 - conventions کہ اُس حصہ کہ مخصوص codebase

~/.claude/CLAUDE.md میں ملتی کیونکہ و project instructions کو team member عام غلطی: ایک نئے میں رکھی گئی تھیں (user-level) ~/.claude/CLAUDE.md کہ بجائے (project-level)

5.2 @path Syntax (File Imports)

و جاتی configuration modular کر سکتا ہے، جس سے refer کہ ذریعہ @path کو files بیرونی CLAUDE.md ہے:

Project CLAUDE.md

میں بیان کیہ گئے ہیں @./standards/coding-style.md
Coding standards
میں ہیں @./standards/testing-requirements.md
Test requirements
میں ہیں @package.json dependencies میں اور @README.md
Project overview

کہ: قواعد @path

- (نہیں space کوئی) آئے file path کہ فوراً بعد @
- Relative اور absolute paths دونوں supported ہیں
- Relative paths کہ مطابق file اُس resolve میں جس میں import ہوتا ہے
- Maximum import nesting depth 5 ہے

شامل ہوتا ہے۔ ہیں standards میں صرف متعلقہ package کم ہوتی ہے اور duplication اس سے

5.3 .claude/rules/ Directory

کرنے کہ organize کہ حساب سے topic کو rules کا متبادل ہے، جو monolithic CLAUDE.md .claude/rules/ لیے استعمال ہوتا ہے:

```
.claude/rules/
testing.md          -- testing conventions
api-conventions.md  -- API conventions
deployment.md       -- deployment rules
react-patterns.md   -- React patterns
```

YAML frontmatter کے ساتھ `paths` conditional loading:

```
---
paths: ["src/api/**/*.ts"]
---
```

استعمال کریں `explicit error handling` کے ساتھ `async/await` کہ لیمہ API files
واپس کرہم `endpoint standard response wrapper` مہر

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
```

استعمال ہونہ چاہئیں `describe/it blocks` میں Tests
نہیں `hardcoding` استعمال کریں `Data factories`
استعمال کریں `test database` — نہ کریں `Database mock`

یہ کیسے کام کرتا ہے:

- کرتا match سہ `paths` pattern کرے جو file edit ایسا Claude Code ہوتا ہے جب load صرف اس وقت Rule ہو
- نہیں ہوتے `rules load` بچتے ہیں — غیر متعلقہ `tokens` اور context اس سہ
- میں codebase files کرنے دیتے ہیں، چاہے `conventions apply` کے لحاظ سہ `file type` آپ کو `Glob patterns` (کے لیے بہت مفید tests) کے ہیں بھی ہوں

کب استعمال کریں `directory-level CLAUDE.md` بمقابلہ `.claude/rules/` والے `paths`:

- `.claude/rules/` with `paths` — جب `conventions` کئی directories میں بکھری files (tests, migrations) پر لاگو ہوں
- `Directory-level CLAUDE.md` — جب `conventions` کسی خاص directory اور کے ہیں ہوں وابستہ ہوں اور کے ہیں ضروری نہ ہوں

5.4 Custom Slash Commands اور Skills

skills کو (`.claude/commands/`) custom commands میں Claude Code version نوٹ: موجود
بناتے `commands` `/name` formats کر دیا گیا ہے دونوں unify کے ساتھ (`.claude/skills/`)
supported اب بھی format کا ذکر کرتی ہے — `.claude/commands/` میں امتحانی گائیڈ

Slash commands reusable prompt templates میں جو `/name` کے ذریعہ `invoke` کے ہیں:

`.claude/commands/` format (legacy, supported):

```
.claude/commands/  
  review.md      -- /review -- standard code review  
  test-gen.md    -- /test-gen -- test generation
```

`.claude/skills/` format (current):

```
.claude/skills/  
  review/SKILL.md -- /review -- frontmatter configuration  
  test-gen/SKILL.md
```

Project commands (`.claude/commands/` یا `.claude/skills/`):

- کرنے والے سب کے لیے دستیاب ہوتے ہیں repo clone میں محفوظ ہوتے ہیں اور VCS
- یقینی بناتے ہیں consistent workflows ٹیم میں

User commands (`~/.claude/commands/` یا `~/.claude/skills/`):

- انہیں ہوتے ہیں share کے ذریعے VCS جو commands ذاتی
- کے لیے workflows انفرادی

5.5 Skills — `.claude/skills/`

Skills advanced commands میں سے SKILL.md frontmatter کے ذریعے configure کے ہیں جو:

```
---  
context: fork  
allowed-tools: ["Read", "Grep", "Glob"]  
argument-hint: "Path to the directory to analyze"  
---
```

کا تجزیہ کریں۔ code structure میں directory دی گئی
تیار کریں۔ report پر ایک architectural patterns اور Dependencies

Frontmatter parameters:

Parameter	Description
<code>context:</code> <code>fork</code>	نہیں کرتا pollute کو main session میں Verbose output میں چلانا کے isolated subagent کو Skill
<code>allowed-</code> <code>tools</code>	نہیں کرتا skill files delete مثلاً — security) دستیاب ہوں گے tools کے محدود کرتا کے کون سے (سکے گی اگر اجازت نہ ہو)
<code>argument-</code> <code>hint</code>	مانگنے کا اشارہ argument ہو تو invoke کے بغیر skill parameters جب

Skill کے استعمال کریں `CLAUDE.md` بمقابلہ:

- **Skill** — task مخصوص کے لیے on-demand invocation (review, analysis, generation)
- **CLAUDE.md** — general standards اور conventions ہمیشہ لوڈ ہونے والے

Personal skills (~/ .claude/skills/):

- متاثر نہ ہونے teammates مختلف ناموں سے بنائیں تاکہ personal variants اپنی

5.6 Planning Mode بمقابلہ Direct Execution

Planning mode:

- نہیں کرتا changes کرتا ؛ plan اور investigate صرف Model
- استعمال کرتا Read، Grep، Glob کرنے کے لیے Codebase explore
- کرتا approve user تیار کرتا جسے implementation plan ایک
- safe exploration کے side effects بغیر

Planning mode کب استعمال کریں:

- files (درجنوں) changes بڑے
- Multiple plausible approaches (microservices: boundaries کیسے define ہوں؟)
- Architectural decisions (کیا framework کون سا؟ structure؟)
- Unfamiliar codebase (change پر عمل سمجھنا ضروری ہے)
- Library migrations 45+ files کو متاثر کریں

Direct execution کب استعمال کریں:

- کے ساتھ stack trace واضح Single-file fixes
- شامل کرنا validation check ایک
- changes اچھی طرح سمجھ گئے، غیر مبہم

Combined approach:

1. Investigation اور design کے لیے planning mode
2. User plan approve کر
3. Approved plan implement کے لیے direct execution

Explore subagent — codebase explore کے لیے specialized subagent:

- Verbose output کو main context سے isolate کرتا
- واپس کرتا summary صرف
- Multi-phase tasks میں context-window exhaustion روکتا

5.7 /compact Command

/compact context compress کے لیے built-in command ہے:

- میں جگہ خالی کرتا context window کو summarize history چھپی
- بھر جائے context سے verbose tool output میں استعمال ہوتا ہے جب Long investigation sessions
- Risk: exact numeric values، dates، اور specific details summarization میں گم ہو سکتے ہیں

- Long investigations پر پھیلی ہوئی ان کے لیے مفید sessions جو متعدد
- Risk: tool results stale ہوں تو files کے بعد session اگر پچھلی

fork_session shared context سے independent branch بناتا ہے:

```
Codebase investigation
|
fork_session
/      \
Approach A:  Approach B:
Redux       Context API
```

- کرتے ہیں context inherit تک forks branch point دونوں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- Approaches compare یا strategies test کرنے کے لیے مفید

Resume نئی session کے بجائے نئی:

- Tool results stale ہوں (files بدل چکی ہوں)
- ہو گیا وہ context degrade کافی وقت گزر چکا ہو اور
- کرنے کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ یہ ہے": resume کے ساتھ tool data پرانے کیا جائے restart "... سے"

6 Prompt Engineering — Advanced Techniques

دستاویزات: [Prompt Engineering](#) | [Anthropic Cookbook](#)

6.1 Few-shot Prompting

Few-shot prompting میں 2–4 prompt دکھانے کے لیے expected behavior وہ طریقہ ہے جس میں input/output examples شامل کیے جاتے ہیں۔

Few-shot text descriptions سے زیادہ مؤثر کیوں ہے:

- "وہ" کو کئی طریقوں سے سمجھا جا سکتا ہے precise جیسے "زیادہ" instruction میں
- دکھاتا ہے decision logic اور expected format غیر میں طور پر Example
- (دہراتا نہیں examples صرف) کرتا ہے generalize پر cases کو نئے Model pattern

Few-shot examples کی اقسام اور استعمال:

1. Ambiguous scenarios کے لیے examples:

Request: "My order is broken"

Action: Call get_customer -> lookup_order -> check status.

Rationale: "broken" damaged item میں درکار order details ہو سکتا ہے؛ آپ کو

Request: "Get me a manager"

Action: escalate_to_human call کریں فوراً

Rationale: Customer مانگ رہا ہے خود حل کرنے کی کوشش نہ کریں human واضح طور پر

2. Output formatting کے لیے examples:

Finding example:

```
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. Acceptable problematic code کے لیے examples:

```
// Acceptable (flag نہ کریں):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. Different document formats سے extraction کے examples:

Document with inline citations:

"As shown in the study (Smith, 2023), the rate is 42%."

-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:

"The rate is 42%. [1]"

-> {"value": "42%", "source": "reference_1", "type": "bibliography"}

5. Informal measurements کے لیے examples:

Text: "about two handfuls of rice"

-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"

-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}

Few-shot اور non-standard measurement units میں مؤثر جو purely rule-based instructions کے لیے بہت متنوع ہوتی ہیں

Prompts میں format normalization rules: structured output کے لیے strict JSON schemas استعمال شامل کریں normalization rules میں prompt، کریں

Normalization:

- Dates: ہمیشہ ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency": "USD"}
- Percentages: decimal fraction; "half" -> 0.5

وہ values inconsistent ہیں مگر JSON syntactically کم ہیں۔ ان semantic errors اس سے

6.2 Explicit Criteria بمقابلہ Vague Instructions

Bad (vague):

Code comments کی accuracy چیک کریں۔
Conservative high-confidence findings report صرف رہیں۔

Good (explicit criteria):

Comment کو problematic میں اُس صورت میں flag کریں جب:

1. Comment کا actual code behavior سے CONTRADICT ہو کرتا ہو
2. Comment کا حوالہ دہ variable یا function کسی غیر موجود
3. TODO/FIXME comment ایسے bug کی طرف اشارہ کرے جو کسی ایسے

Flag نہ کریں:

- کہ لحاظ سے پرانہ ہوں style جو صرف comments ایسے
- wording inaccuracies معمولی
- Missing comments (category یہ الگ)

Severity criteria examples کے ساتھ define کریں:

CRITICAL: User کے لیے runtime failure
Example: Payment process کرتے وقت NullPointerException

HIGH: Security vulnerability
Example: SQL injection, XSS, missing authorization checks

MEDIUM: Logic bug بغیر فوری impact کے
Example: Wrong sorting, off-by-one error

LOW: Code quality
Example: Duplication, suboptimal algorithm کے لیے data چھوٹے

6.3 Prompt Chaining

Prompt chaining میں توڑتا sequence کی focused steps کو ایک complex task

Step 1: auth.ts analyze کریں (صرف local issues)

-> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)

-> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)

-> Output: module boundaries پر issues

یہ کیوں اہم ہے:

- **Attention dilution** — جب files ایک ساتھ دیکھتا ہے تو کچھ files ایک ساتھ دیکھتا ہے، model سے بچاتا ہے — جب
- دیکھتا ہے files دیکھتا ہے تو کچھ files ایک ساتھ دیکھتا ہے، model سے بچاتا ہے — جب
- consistent analysis quality — لیکن analysis quality
- Cross-file interactions — الگ analysis

کب استعمال کریں dynamic decomposition بمقابلہ Prompt chaining

- **Prompt chaining** — predictable, repeatable tasks (code review, file migrations)
- **Dynamic decomposition** — open-ended investigations ان subtasks execution کے دوران واضح ہیں

6.4 “Interview” Pattern

Solution implement کرنے کے لیے Claude clarifying questions پوچھتا ہے:

Claude: "API کے لیے caching implement کرنے سے پہلے چند سوالات"

1. پسند کرتے ہیں؟ cache invalidation strategy میں سے کون سی event-based یا TTL آپ
2. قابل قبول ہے؟ stale data ہو تو کیا cache unavailable اگر
3. Caching per-user یا global؟
4. Cache کے لیے expected data volume کیا ہے؟

یہ کب مفید ہے:

- Unfamiliar domain (fintech, healthcare, legal systems)
- Implications tasks (cache strategies, failure modes) واضح
- پر منحصر context ان بہترین انتخاب viable approaches متعدد

6.5 Validation اور Retry-with-Feedback

کر: extracted data validation fail

Step 1: Document سے data extract کریں

Step 2: Validate کریں (Pydantic, JSON Schema, business rules)

Step 3: retry کریں کہ ساتھ context — error اگر

- Original document
- Previous (incorrect) extraction
- Specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."

کب مؤثر ہوگا Retry:

- Format errors (date غلط format میں)
- Structural errors (field جگہ غلط)
- Arithmetic inconsistencies (model check دوبارہ کر سکتا ہے)

Retry گا کہ مدد نہ میں کرے گا:

- Information source document میں موجود ہے نہ میں
- Required context external (data کسی اور document میں کیا گیا)

Pydantic validation tool: Pydantic Python library جو schema-based data validation کے لیے استعمال ہوتی ہے۔ امتحان کے لیے اس کے نکات یہ ہیں:

- **Structural validation:** types, requiredness, enum constraints JSON کے بعد code میں جانے دیے جاتے ہیں
- **Semantic validation:** custom validators business logic enforce کرتے ہیں (sum of items equals total! start_date < end_date)
- **Validate-retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context میں prompt ساتھ دوبارہ کریں
- **JSON Schema generation:** Pydantic models سے JSON Schema generate کر سکتے ہیں۔ `tool_use` کے لیے single source of truth میں، جو ایک فراہم کرتا ہے

6.6 Self-correction

Internal contradictions detect کرنے کا pattern:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

Model stated value اور computed value میں تو مختلف ہیں — اگر وہ `conflict_detected` آپ دونوں نکالتا ہے —

7 باب: Message Batches API

دستاویزات: [Message Batches](#)

7.1 Overview

Message Batches API کو asynchronous processing کے لیے requests کے batches submit دینا ہے:

Attribute	Value
Savings	50% مقابلہ میں synchronous calls
Processing window	24 hours (latency SLA guarantee) زیادہ سے زیادہ
Multi-turn tool calling	Supported (request = ایک response = ایک)
Correlation	request اور response کو جوڑنے کے لیے custom_id field

7.2 Batch API Synchronous API بمقابلہ کب استعمال کریں

Task	API	Why
Pre-merge PR check	Synchronous	Developer 24 hours قابل قبول نہیں انتظار کر رہا ہے
Overnight tech-debt report	Batch	Result 50% savings صبح تک چاہیے
Weekly security audit	Batch	50% savings فوری نہیں
Interactive code review	Synchronous	درکار response فوری
10,000 documents process کرنا	Batch	Bulk processing! savings میں

7.3 custom_id کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

آپ کو یہ کرنے دیتا ہے custom_id:

- Result کو original document link سے
- Failure کو submit دوبارہ failed documents کی صورت میں صرف
- Successful documents کو process دوبارہ

7.4 Batch میں Failures کو Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed 5 fail (context limit exceeded) ہوں
3. Failures کو custom_id سے identify کریں
4. Strategy میں تقسیم کریں chunks کو long documents (مثلاً) بدلیں

کریں submit کو دوبارہ failed documents صرف اُن 5.5

7.5 SLA Planning

تک لگ سکتے ہیں hours کو 24 Batch API چاڑھ اور result میں hours اگر آپ کو 30

- Submission window: 30 - 24 = **6 hours**
- کریں submit پڑھ hours کم از کم 24 deadline کو Batches
- میں تقسیم کریں 4-hour windows لیں 4 Frequent submissions

8 Task Decomposition Strategies

8.1 Fixed Pipelines (Prompt Chaining)

ہوتا ہے defined پڑھ step کر

Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output

کب استعمال کریں:

- Task structure predictable ہو (reviews ایک ہی template follow کریں)
- پڑھ سہ معلوم ہوں steps تمام
- چاڑھ reproducibility اور stability آپ کو

8.2 Dynamic Adaptive Decomposition

ہوتا ہے ہیں generate کی بنیاد پر Subtasks intermediate results

- "Legacy codebase لیں tests add کریں"
- > Glob, Grep کریں structure map پڑھ
- کہ ساتھ partial coverage کہ، 2 tests بغیر modules ملا: 3 ->
- > Prioritize: payments module شروع کریں (high risk)
- دریافت ہوئی dependency پر external API کام کہ دوران ->
- > Adapt: tests پڑھ mock add کریں کہ لیں external API لکھنے سہ پڑھ

کب استعمال کریں:

- Open-ended investigative tasks
- شروع میں معلوم نہ ہو scope جب مکمل
- پر منحصر ہو results کہ step پچھلا step جب کر

8.3 Multi-pass Code Review

کے لیے pull requests والی 10+ files

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files درمیان کہ relationships analyze کریں
-> Cross-file issues: inconsistent types, circular dependencies
```

کیوں خراب pass پر ایک سی 14 files:

- Attention dilution: shallow کچھ کی، deep analysis کی files کچھ
- Inconsistent comments: approve دوسری میں، pattern flag میں ایک file
- Missed bugs: errors کی وجہ سے واضح cognitive overload

9 Escalation اور Human-in-the-Loop: باب 9

9.1 کریں Escalate کی طرف Human کب

Escalation triggers (rules واضح):

Situation	Action
Customer "get me a manager" واضح طور پر کہہ	کریں؛ حل کرنے کی کوشش نہ کریں escalate فوراً
Policy request نہ کرتی ہو cover کو	Escalate policy جب competitor price matching (مثلاً) کریں (ہو)
Agent پیش رفت نہ کر سکا	کریں escalate کہ بعد attempts مناسب تعداد میں
Threshold financial operation اوپر	Escalate enforce، prompt کہ ذریعہ hook ترجیحاً) کریں (نہیں)
Customer multiple matches تلاش کرتے وقت	مانگیں؛ انداز نہ لگائیں identifiers مزید

نہیں ہیں reliable trigger جو:

Unreliable method	Why it fails
Sentiment analysis	نہیں کرتا correlate کہ mood case complexity Customer
Model self-rated confidence (1–10)	کمزور calibration غلط ہو سکتا ہے؛ Model confidently
Automatic classifier	موجود نہ ہو؛ training data؛ ممکن Overengineering

9.2 Escalation Patterns

Immediate escalation:

Customer: "I want to speak to a manager"
Agent: [کرتا == escalate_to_human call فوراً]
NOT: "I can help with your issue, let me..."

Resolve escalation: کی کوشش کے بعد

Customer: "My refrigerator broke two days after purchase"
Agent: [کرتا == warranty replacement offer، چیک کرتا == order]
escalate -> مطمئن نہ ہو customer اگر

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [frustration acknowledge] "سمجھتا ہوں frustration میں آپ کی"
[resolution offer] "پیش کر سکتا ہوں refund یا replacement میں"
Customer: "No, I want to talk to someone!"
Agent: [customer دوبارہ insist کرہ -> immediate escalation]

کریں escalate دیں، اور صرف اسی وقت concrete solution کریں، پھر emotion acknowledge بنیادی اصول: پل۔
manager (ی۔) نہ کریں escalate کے پل۔ اطلاع پر dissatisfaction کی خواہش دہرائیں customer human جب
(مانگنے کے برابر نہیں ہیں)

Policy gap کے لیے escalation:

Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: کرتی == price adjustments cover پر site صرف اپنی
Agent: [escalate – policy competitor price matching کو cover نہیں کرتی]

9.3 Structured Handoff Protocols

کو دینی چاہیے structured summary human agent پر Escalation:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement – customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

دیکھنا summary تک رسائی نہیں ہوتی — صرف یہ full conversation transcript کو Human operator

ہونی چاہیے self-contained اس لیے یہ مکمل اور

9.4 Confidence Calibration اور Human Oversight

Data extraction systems □J □K:

1. **Field-level confidence scores:** model extracted field confidence score نکالنا
2. **Calibration:** labeled validation sets استعمال کر کے thresholds tune کریں
3. **Routing:**
 - High confidence + stable accuracy -> automated processing
 - Low confidence & ambiguous sources -> human review

Stratified random sampling:

- High-confidence extractions □ sample audit □ لی□ بھی باقاعدگی سد
- Aggregate 97% accuracy □ errors □□ چھپا سکتی □□ document type 40 لی□ خصوص
- Accuracy □□ analyze □□ حساب □□ field اور □□ document type □□، overall کو صرف

10 Error Handling میں Multi-agent Systems: پایب

10.1 Error Categories

Category	Examples	Retryable	Agent action
Transient	Timeout, 503, network failure	Yes	Exponential backoff retry ساتھ
Validation	Invalid input format, missing required field	No (input fix کریں)	Request modify کریں اور retry کریں

Business	Policy violation, threshold exceeded	No	User alternative کو سمجھائیں؛ User کریں
Permission	Access denied	No	Escalate کریں

10.2 Error-handling Anti-patterns

Anti-pattern	Problem	Correct approach
Generic status “search unavailable”	Coordinator decide نہیں کر سکتا کہ recover کیسے کرنا ہے	Error type, query, partial results, alternatives واپس کریں
Silent suppression (empty result = success)	Coordinator match سمجھتا ہے کہ failure ملا، حالانکہ	“no results” اور “search failure” میں فرق کریں
Workflow پر پورا failure ایک abort کرنا	ضائع ہو جاتا ہے partial results تمام	Partial results کے ساتھ جاری رکھیں؛ gaps annotate کریں
Subagent میں infinite retries	Latency اور resources کا ضیاع	Local recovery (1–2 retries)، پھر coordinator تک propagate

10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    { "title": "AI Music Generation Report", "url": "...", "relevance": 0.8 }
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

کو فیصلہ کرنے کے لیے ضروری معلومات دیتا ہے coordinator

- Modified query کے ساتھ retry کریں؟
- Partial results استعمال کریں؟
- delegate کو subagent کسی اور کریں؟
- gap annotate کے بغیر جاری رکھیں اور اس section

10.4 Final Synthesis میں Coverage Annotations

Report: AI Impact on Creative Industries

Visual Art (FULL COVERAGE)

[research results]

Music (PARTIAL COVERAGE – search agent timeout)

[partial results]

⚠ Note: اس section کی coverage search agent timeout کی وجہ سے محدود ہے۔

Literature (FULL COVERAGE)

[research results]

11 Context میں Production Systems: باب 11 Management

11.1 Facts کو Separate Block میں Extract کریں

key facts (جو summarization کے دوران degrade ہو جاتی ہیں) پر انحصار کرنے کے بجائے conversation history کو structured block میں extract کریں:

```
=== CASE FACTS (کیا جائے update آئے fact جب بھی نیا) ===  
Customer ID: CUST-12345  
Order ID: ORD-67890  
Order Date: 2025-01-15  
Order Amount: $89.99  
Issue: Damaged item on delivery  
Customer Request: Full refund  
Status: Pending manager approval  
===
```

کیوں نہ ہو summarized history میں شامل کریں، چاہے prompt کے block یں

11.2 Tool Results کو Trimming کرنا

کے لیے صرف 5 درکار ہیں current task واپس کرتا ہے مگر 40+ fields lookup_order اگر

```
# PostToolUse hook: رکھیں relevant fields صرف
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

کم ہوتی noise بچتا اور context اس سے

11.3 Position-aware Input

Lost-in-the-middle effect رکھیں critical information کو ذہن میں رکھتے ہوئے:

```
[KEY FINDINGS – اوپر]
3 critical vulnerabilities ملیں...

[DETAILED RESULTS – درمیان]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS – آخر میں]
Priority: merge auth.ts vulnerabilities fix کریں.
```

11.4 Scratchpad Files

Long investigations میں agent key findings scratchpad file لکھ سکتا ہے:

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() موتی ۵۵ call سے places تین OrderController, AdminPanel, CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 نہ refund_reason (NOT NULL) add 01-12-2024 کیا
```

scratchpad دوبارہ چلانے کے بجائے agent discovery (میں session یا نئی) کو جائزہ context degrade کر سکتا ہے

11.5 Context Subagents کو Delegate کرنا

Main agent: "payments module کی dependencies investigate کریں"

-> Subagent (Explore): 15 files read کرتا، imports trace کرتا

-> Returns: "Payments depends on AuthService, OrderModel, and the external PaymentGateway API"

Main agent: context 15 files میں رکھتا ہے line کہ بجائے ایک

Separate context layer: Multi-agent systems میں subagent context budget — میں کام کرتا ہے کہ طور پر separate context layer ایک Coordinator ملتی ہے کہ information کہ لی ضروری task اسے صرف اپنے کرتا context allocate کرتا ہے، اور global state store، کرتا ہے subagent outputs aggregate کرتا ہے: وہ کہ سہا بہر دیتا ہے کہ جو information کو ایسی agent window نہیں کہتا، جہاں ایک "context leakage" کہ اس سہا دوسروں کہ لی کہ غیر متعلق کہو

Subagents کہ constrained context budgets:

- Minimal context: specific task + ضروری data بھیجیں
- Subagent کو raw data dumps کہ بجائے structured results کی ہدایت دیں
- اور کم distractions کا مطلب کم tools کریں — کم limit کو toolset کہ subagent کہ allowedTools کہ context cost

11.6 Structured State Persistence (crash recovery کہ لی کہ)

کرتا ہے کہ export کہ location ایک معلوم state اپنی agent کہ

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

Coordinator resume کہ manifest load کہ کرتا ہے:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

کو محفوظ رکھنا Provenance: باب 12

12.1 Attribution Loss Problem

گم ہو سکتا ہے link "claim → source" کی جانچ میں، تو results summarize سے sources جب متعدد

Bad: "The AI music market is estimated at \$3.2B." (No source, no year.)

Good:

```
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 Conflicting Data کو Handle کرنا

دیں values مختلف sources جب دو

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period کا فرق"
}
```

کو coordinator کے ساتھ محفوظ رکھیں اور attribution کو اندھا دھند منتخب نہ کریں۔ دونوں کو value کسی ایک فیصلہ کریں دیں۔

12.3 Correct Interpretation کے لیے Dates شامل کریں

سمجھ لیا جا سکتا ہے contradiction کو temporal differences کے بغیر Dates

Bad: "Source A says 10%, source B says 15%. Contradiction."

Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ % growth."

12.4 Content Type مطابق Render کریں

میں نڈ ٹھونسیں format ر چیز کو ایک ی:

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

13 Claude Code کے Built-in Tools: باب 13

13.1 Tool Selection Reference

Task	Tool	Example
تلاش کرنا files سڈ pattern نام	Glob	<code>**/*.test.tsx</code> , <code>src/components/**/*.ts</code>
File contents اندر search کے	Grep	Function name, error message, import
File مکمل پڑھنا	Read	Analysis کے لیے file load کرنا
بنانا file نیا	Write	Scratch سڈ file create کرنا
precise edit میں file موجود	Edit	Unique text match کے ذریعے specific snippet replace کرنا
چلانا Shell command	Bash	git, npm, tests, build

13.2 Incremental Investigation Strategy

نڈ پڑھیں سمجھ بوجھ کو رفتہ رفتہ بنائیں files ایک ساتھ تمام:

- Grep: entry points تلاش کریں (function definition, export)
- Read: پڑھیں files ملنے والی
- Grep: usages تلاش کریں (import, calls)
- Read: consumer files پڑھیں
- کریں repeat نہ مل جائے picture جب تک مکمل

13.3 Fallback: Edit کے بجائے Read + Write

و جائے fail کی وجہ سے Edit non-unique text match جب

1. Read — لوڈ کریں file content مکمل
2. Content کو programmatically modify کریں
3. Write — updated version لکھ دیں

امتحانی ڈومین نوٹس: II حصہ

1 Agent Architecture اور Orchestration ڈومین: 1 (27%)

1.1 Autonomous Task Execution اور Agentic Loops Design

Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` ("tool_use" بمقابلہ "end_turn") چیک واپس کریں results کو iteration کے لیے اگلی tools execute کریں،
- Tool results conversation history میں append کریں تاکہ model کو action اگلا model کو history میں
- Model-driven decision making (Claude tool اگلا tool) بمقابلہ hard-coded decision trees

Key skills:

- Flow control: جب `stop_reason = "tool_use"` تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations درمیان context کو tool results میں append کرنا
- Anti-patterns بچنا: completion کے لیے assistant text parse کرنا، primary stopping mechanism پر استعمال کرنا arbitrary iteration limits

1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate

Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، routing کا مالک ہوتا ہے
- Subagents isolated context میں کام کرتے ہیں — coordinator کی history inherit نہیں کرتے
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کا خطرہ overly narrow decomposition

Key skills:

- Research coverage کو subagents میں تقسیم کریں تاکہ duplication کم ہو

- Iterative refinement loops implement کریں (coordinator synthesis evaluate اور tasks re-route کریں)
- Observability تمام communication coordinator کے ذریعے route کے لیے تمام

1.3 Subagent Calls, Context Passing, اور Spawning Configure کرنا

Key knowledge:

- Task tool subagents spawn؛ coordinator کے allowedTools میں "Task" ہونا چاہیے
- Subagent context prompt میں explicitly؛ ہونا چاہیے
- AgentDefinition configuration: descriptions, system prompts, tool constraints
- Alternatives explore کے لیے fork_session کے ذریعے session management

Key skills:

- Previous agents کے full outputs subagent prompt میں شامل کریں
- Context pass کے structured formats کے لیے separate کو metadata اور data کرتے وقت
- Coordinator turn میں متعدد Task calls کے parallel subagents spawn کے ذریعے
- Coordinator prompts goals اور quality criteria کے طور پر لکھیں، step-by-step instructions کے طور پر نہیں

1.4 Enforcement اور Handoff Patterns کے ساتھ Multi-step Workflows Implement کرنا

Key knowledge:

- Workflow ordering کے لیے programmatic enforcement (hooks, preconditions) اور prompt guidance کے درمیان فرق
- deterministic guarantees کے وں درکار (مثلاً مالی operations کے identity verification)، prompts کافی نہیں ہوتے
- Escalation کے دوران structured handoff protocols (customer ID, reason, recommended action)

Key skills:

- مکمل کے steps کریں جب تک کہ downstream calls کو block نہ لیں (مثلاً process_refund block کریں واپس کرنے تک verified ID کے get_customer مثلاً)
- Multi-aspect customer requests کے items کو الگ میں تقسیم کریں
- Human کو escalate کے وقت structured summaries produce کریں

1.5 Tool Calls Intercept اور Data Normalize کرنے کے لیے Agent SDK Hooks

Key knowledge:

- Hook patterns (مثلاً PostToolUse) tool results کو model کے consume کے لیے intercept کرنے کے لیے

- Hooks outgoing calls intercept ☐ کریں ☐ compliance rules enforce ☐ (مثلاً threshold اوپر refunds سدا کرنا)
- Hooks **deterministic guarantees** ☐ جبکہ ☐ prompt instructions **probabilistic compliance** دیتی ☐ ہیں

Key skills:

- `PostToolUse` hooks `data` formats normalize کریں (Unix timestamps, ISO 8601, numeric status codes)
- Policy-violating actions block `interception` hooks اور `escalation` استعمال کریں اور `redirect` کریں
- منتخب کریں `hooks` `prompts` `business rules` کو `guaranteed compliance` کو تو

1.6 Complex Workflows Task Decomposition Strategies

Key knowledge:

- **Fixed pipelines** (prompt chaining) □ بمقابلہ intermediate results کی بنیاد پر **dynamic adaptive decomposition**
- Prompt chaining: sequential steps (□ file کو analyze پھر integration pass کریں)
- Adaptive investigation plans □ ہیں generate subtasks کی بنیاد پر discovered information جو

Key skills:

- Predictable multi-aspect reviews □ ک prompt chaining؛ open-ended investigations □ لی dynamic decomposition
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade □ لی□ لی structure map پھر prioritized plan بنائیں

1.7 Session State, Resuming, اور Forking

Key knowledge:

- Named sessions continue `--resume <session-name>` کرنا کے لیے
- Shared context `fork_session` بنانا کے لیے independent investigation branches سے
- Resume بتانا کی اہمیت file changes کو agent کو وقت
- Structured summary `reliable` زیادہ سے زیادہ `resuming` کے ساتھ `session` کے ساتھ نئی

Key skills:

- Named investigation sessions continue `--resume` کریں
- Approaches parallel compare `fork_session` کریں
- Resuming (context `current` اور نئی `session` شروع کریں) (results stale ہیں) `fork_session` کریں

5.10 fork_session اور Session Management

کرتا --resume <session-name> named session کو resume

```
claude --resume investigation-auth-bug
```

- جاری رکھتا conversation کے ساتھ بچھلی saved context
- پر پھیلی ہوں ان کے لیے مفید sessions جو متعدد Long investigations
- وہ ہو سکتے ہیں tool results stale بدل چکی ہوں تو files کے بعد session اگر بچھلی Risk:

fork_session shared context سے independent branch بناتا

```
Codebase investigation
  |
  fork_session
  /      \
Approach A:  Approach B:
Redux       Context API
```

- کرتے ہیں context inherit تک forks branch point دونوں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- کرنے کے لیے مفید strategies test کرنے یا Approaches compare

Resume نئی session کے بجائے نئی

- Tool results stale ہوں (files بدل چکی ہوں)
- وہ ہو گیا context degrade کافی وقت گزر چکا ہو اور
- کرنے کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ یہ ہے: resume کے ساتھ tool data پرانے کیا جائے restart "... سے

2 Tool Design اور MCP Integration (18%)

2.1 Clear Descriptions کے ساتھ Tool Interfaces Design کرنا

Key knowledge:

- Tool descriptions minimal descriptions کرتا ہے؛ LLM tools select میں جن سے mechanism و بنیادی unreliable selection دیتی
- شامل کرنے کی اہمیت applicability boundaries اور Input formats, example queries, edge cases
- Ambiguous یا overlapping descriptions misrouting میں سبب بنتی
- بنا سکتی ہے associations کے ساتھ غیر ارادی System prompt wording tools

Key skills:

- کریں distinguish سے واضح طور پر similar alternatives کو tool لکھیں جو ر descriptions ایسی

- Functional overlap (مثلاً) `analyze_content` -> `extract_web_results` tools rename کریں ختم کرنے کے لیے
- واضح input/output contracts میں تقسیم کریں جن کے specialized tools کو General-purpose tools

2.2 MCP Tools کے Structured Error Responses Implement کرنا

Key knowledge:

- MCP tool responses میں `isError` flag
- **Transient errors** (timeouts), **validation errors** (bad input), **business errors** (policy violations), اور **access/permission errors** کے درمیان فرق
- Generic errors ("Operation failed") میں recovery decisions روک دیتے ہیں صحیح
- Retryable اور non-retryable errors میں فرق

Key skills:

- `errorCategory` (transient/validation/permission), `isRetryable` اور human-readable message جیسی structured metadata واپس کریں
- Business-rule violations کے ساتھ user-facing explanations کے لیے واضح `retryable: false` استعمال کریں
- Transient failures کے لیے subagents میں local recovery صرف وہ errors propagate کریں؛ صرف وہ حل نہ کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

Key knowledge:

- کم کرتی tool selection reliability (مثلاً 18 کے بجائے 4-5) tools بڑھاتے ہیں
- Specialization کے لیے agents رکھنے والے tools سے باہر
- Scoped tool access: role-relevant tools + محدود cross-role utilities
- `tool_choice: "auto"`, `"any"` اور forced tool selection (`{"type": "tool", "name": "..."}`)

Key skills:

- تک محدود رکھیں relevant tools صرف subagent کے toolset
- General tools کو constrained alternatives سے بدلیں (مثلاً `fetch_url` -> `load_document`)
- یقینی tool call کے بجائے text answer استعمال کریں تاکہ `tool_choice: "any"`
- Specific tool force execution order assured کریں تاکہ

2.4 MCP Servers اور Claude Code میں Integrate کرنا

Key knowledge:

- MCP server scope: user کے لیے experiments بمقابلہ (`.mcp.json`) project ٹیموں کے لیے (`~/claude.json`)
- Secrets management کے لیے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}`)
- available ہوتے ہیں اور ایک ساتھ discover پر tools connection کے connected MCP servers تمام ہیں
- MCP resources بطور “content catalogs” (task summaries, database schemas) exploratory tool calls کم کرتے ہیں

Key skills:

- Shared MCP servers کو project `.mcp.json` میں env-var-based tokens ساتھ configure کے کریں
- Personal/experimental servers کو `~/claude.json` میں رکھیں
- Standard integrations کے لیے community MCP servers کو custom servers پر ترجیح دیں

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور Apply کرنا

Key knowledge:

- **Grep**: file contents اندر search (function names, error messages, imports)
- **Glob**: name/extension patterns سے files تلاش کرنا
- **Read/Write**: full-file operations؛ **Edit**: unique text matches کے ذریعے precise changes
- **Read + Write** پر واپس جائیں تو fail کی وجہ سے Edit non-unique matches

Key skills:

- Content search کے لیے Glob کے لیے pattern-based discovery اور Grep کے لیے
 - Read کرنے کے لیے flows trace پھر Grep کے لیے entry points: سمجھ بوجھ رفتہ رفتہ بنائیں
 - Wrapper modules کے ذریعے function usage trace کریں
-

3 اور Claude Code Configuration ڈومین: Workflows (20%)

3.1 Hierarchy, Scope, اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

Key knowledge:

- CLAUDE.md hierarchy: user (`~/ .claude/CLAUDE.md`), project (`.claude/CLAUDE.md` یا root `CLAUDE.md`), اور directory-level (subdirectories میں `CLAUDE.md`)
- User-level settings میں `user` اور `share` کے ذریعے VCS پر لاگو ہوتے ہیں اور صرف ایک `CLAUDE.md` بناتے ہیں
- External files refer (مثلاً `@./standards/coding-style.md`) syntax `@path` کے لیے `CLAUDE.md` modular بنانے کے لیے
- Monolithic CLAUDE.md کے بجائے `.claude/rules/` directory کے لیے topic-focused rule files بنانے کے لیے

Key skills:

- Hierarchy issues diagnose کریں (team member کو instructions دینے میں) اور user-level میں تھیں
- `@path` کے لیے standards selectively include `CLAUDE.md` میں (مثلاً `@./standards/testing.md`) استعمال کریں
- `.claude/rules/` files (testing.md, api-conventions.md, deployment.md) کو متعدد `CLAUDE.md` بڑے میں تقسیم کریں

3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

Key knowledge:

- `~/ .claude/commands/` بمقابلہ `.claude/commands/` میں Project commands (VCS کے ذریعے shared) اور user commands
- `.claude/skills/` میں SKILL.md frontmatter: `context: fork`, `allowed-tools`, `argument-hint`
- `context: fork` skill کو isolated subagent context کے لیے تاکہ main session pollute نہ ہو
- Personal skill variants `~/ .claude/skills/` میں رکھ سکتے ہیں

Key skills:

- Project slash commands `.claude/commands/` میں رکھیں تاکہ پوری team حاصل کرے
- Verbose output کے لیے skills isolate `context: fork` استعمال کریں
- Skill کے tools کے لیے `allowed-tools` محدود کرنے کے لیے استعمال کریں
- Required parameters کے لیے `argument-hint` مانگنے کے لیے استعمال کریں

- کب sequentially اور (interdependent) میں کب دیں message ایک issues تمام

Key skills:

- Transformation requirements 3–2 لی کرنا concrete input/output examples کریں
- Implementation test کے ساتھ performance requirements اور edge cases، expected behavior کے لیے بنائیں
- Design aspects (cache invalidation، failure modes) کے لیے استعمال کریں interview pattern سامنے لانے کے لیے
- Edge cases کے لیے concrete test cases کے ساتھ sample inputs اور expected outputs

3.6 Claude Code میں Integrate CI/CD Pipelines کو

Key knowledge:

- Automated pipelines میں non-interactive mode کے لیے flag (یا `--print`) `-p`
- CI میں structured output کے لیے `--output-format json` اور `--json-schema`
- CLAUDE.md project context (testing standards، review criteria) فراہم کرنا
- **Session context isolation:** code generate والی review اسے same session کرنے والی کوئی review

Key skills:

- Interactive input پر hang کے لیے CI کو Claude Code میں `-p` کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے `--output-format json` + `--json-schema` استعمال کریں
- new/unfixed issues (صرف) شامل کریں review results کے وقت پچھلے run کے بعد دوبارہ commits نہ کریں
- Test generation quality کے لیے available fixtures اور testing standards میں CLAUDE.md کے لیے document کریں
- style نہ ہو اور duplication میں شامل کریں تاکہ context کو existing test files کے وقت tests generate نہ کرے

4 Prompt Engineering اور Structured Output (20%)

4.1 Accuracy کے Prompts کے ساتھ Explicit Criteria، تر کرنے کے لیے کرنا Design

Key knowledge:

- Explicit criteria vague instructions میں زیادہ مؤثر ہوتے ہیں (مثلاً "flag comments only when they contradict code" بمقابلہ "check comment accuracy")
- "be more conservative" کے مؤثر generic guidance concrete categorical criteria

- درست high false-positive rates میں categories پر اثر انداز ہوتے ہیں: کچھ False positives developer trust کم کر دیتی ہیں trust پر بھی categories

Key skills:

- Review criteria define کرنا (minor style) ignore اور کیا (bugs, security) کرنا report کریں: کیا
- High false-positive rates والی categories طور پر disable عارضی
- کریں explicit severity criteria define کے ساتھ code examples کے لیے level پر

4.2 Output Consistency Few-shot Prompting بہتر کرنے کے لیے استعمال کرنا

Key knowledge:

- Few-shot examples consistently formatted, actionable output کا سب سے مؤثر method پیدا کرنے کا سب سے
- Few-shot cases (tool selection, test coverage gaps) کو دکھا سکتا ہے
- Few-shot model میں defaults میں مدد دیتا ہے، صرف generalize پر patterns کو نئے Few-shot model
- Few-shot extraction tasks میں hallucinations کو کم کر سکتا ہے

Key skills:

- Ambiguous scenarios کے لیے targeted examples کے ساتھ 4–2 rationale
- Output format (location, issue, severity, suggested fix) شامل کریں examples دکھانے والے
- میں فرق دکھائیں real issues اور acceptable code patterns دیں جو examples ایسے
- دیں examples کے extraction سے درست documents والے structures مختلف

4.3 JSON Schemas کے ساتھ tool_use اور Structured Output کرنا Enforce

Key knowledge:

- JSON syntax errors کی ضمانت دینے اور tool_use with JSON Schemas schema-conformant output طریقہ کے سب سے reliable کرنے کا سب سے
- کرنی ہوتی tool call میں اسے لازماً "any" واپس کر سکتا ہے؛ model text میں tool_choice: "auto" منتخب کرتی ہے forced selection specific tool
- Strict JSON Schemas syntax errors سے روکتیں semantic errors ختم کرتی ہیں مگر (totals؛ میں ملے؛ values غلط fields غلط)
- Schema design: required vs optional fields؛ enums کے ساتھ "other" + detail string extensibility کے لیے

Key skills:

- JSON Schemas کے ساتھ extraction tools define کریں اور tool_use results سے data parse کریں
- استعمال کریں tool_choice: "any" یقینی بنانے کے لیے structured output کے ساتھ تو schemas متعدد
- Specific tool call force کریں: tool_choice: {"type": "tool", "name": "extract_metadata"}
- Source information میں fields کے لیے values fabricate نہ ہونے کی صورت میں optional/nullable رکھیں

- Extensible categorization `enum` values + detail fields استعمال کریں اور `"unclear"` اور `"other"` کے لیے

4.4 Extraction Quality ☐ لی ☐ Validation, Retries, اور Feedback Loops Implement کرنا

Key knowledge:

- Retry-with-error-feedback: concrete validation errors میں retry prompt کریں لیکن correction guide شامل کریں
- فوائد:
 - retries میں موجود نئی information source تو
- Feedback loop design: finding trigger والا pattern (detected_pattern) track کریں
- Semantic errors (totals reconcile نہیں ہوتے) syntax errors (جو tool_use address نہیں ہوتے) بمقابلہ

Key skills:

- دیں follow-up prompts کے ساتھ specific validation errors اور، incorrect extraction، Original document،
- تو اس کی شناخت کریں (میں کے اور external document صرف required info کے فوائد کے و retry جب
- False positives analyze کے لیے detected_pattern fields میں findings کرنے کے لیے
- Discrepancies detect کے لیے calculated_total اور stated_total دونوں extract کے کر کے self-correction design کریں

4.5 Efficient Batch Processing Strategies Design کرنا

Key knowledge:

- Message Batches API: 50% savings, 24-hour processing window, latency SLA guarantee
- Batch processing non-blocking tasks (overnight reports, audits) اور blocking tasks (pre-merge checks) کے لیے مناسب
- Batch API single request میں multi-turn tool calling support کرتا ہے
- `custom_id` fields batch request/response correlate کے اندر

Key skills:

- استعمال Batch API کے لیے overnight/weekly workloads اور synchronous API کے لیے blocking checks کریں
- SLA needs 24-hour guarantee (مثلاً 30) کریں batch submission cadence plan کے مطابق
- Failed documents (identified by `custom_id`) submit صرف انہی کو دوبارہ کریں failures handle کر کے
- Large-scale processing کے ساتھ sample سے بڑے prompts iterate کریں

4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

Key knowledge:

- Self-review limitations: model کا challenge ساتھ رکھنا اور اپنے فیصلوں کو reasoning context اپنی model امکان کم ہوتا ہے
- Independent review instances (generation context) subtle issues میں بہتر ہوتی ہیں (کم بغیر generation context)
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچانا

Key skills:

- Changes review استعمال second independent Claude instance کے بغیر generation context کے لیے کریں
- Multi-file reviews cross-file dataflow میں تقسیم کریں تاکہ per-file passes + integration passes analysis ہو سکے
- Self-rated confidence route طریقہ سے calibrated reviews استعمال کر کے verification passes کے ساتھ کریں

5 Context Management اور Reliability (15%) ڈومین 5

5.1 Critical Information Conversation Context محفوظ رکھنا اور Manage کرنا

Key knowledge:

- Progressive summarization کے خطرات: numeric values, percentages, اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable سے کر سکتے ہیں، لیکن درمیان کے findings miss ہیں
- Tool outputs relevance میں context کے مقابلہ میں (40+ fields جب جمع ہو سکتے ہیں) disproportionately میں
- Subsequent API requests کی اہمیت full conversation history میں

Key skills:

- Transactional facts کو summarized history سے باہر ایک persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data کے شروع سے explicit section headings کے ساتھ رکھیں
- Subagents کے structured outputs میں metadata (dates, sources) شامل کروائیں

5.2 Effective Escalation Patterns Design اور Ambiguity Resolve کرنا

Key knowledge:

- Suitable escalation triggers: human ☐ explicit request, policy gaps/exceptions, ☐ واپانا
- Immediate escalation (explicit request) ☐ attempt-to-resolve (agent scope ☐ اندر بمقابلہ)
- Sentiment analysis اور model confidence self-ratings case complexity ☐ unreliable proxies ☐ ہیں
- Multiple customer matches ☐ heuristic guessing ☐ بجائے additional identifiers ☐ چاہئیں مانگندہ ☐ لیے

Key skills:

- System prompt میں explicit escalation criteria few-shot examples ساتھ دیں
- Human execute کو فوراً investigation کو بغیر اضافی explicit request کی
- کربس escalate کو تو silent یا ambiguous کو لی specific request کسی policy جب
- Tool results multiple matches میں additional identifiers کو تو

5.3 Multi-agent Systems میں Error Propagation Strategies Implement کرنا

Key knowledge:

- Structured error context (failure type, query, partial results, alternatives) coordinator کو smarter recovery دیتا □□
- Access failures (timeouts retry decision میں □□□) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses (“search unavailable”) coordinator □□ valuable context میں چھپا دیتا □□
- Silent suppression □□ anti-patterns کرنا دونوں workflow abort پر پورا failure یا ایک

Key skills:

- Structured error context failure type, attempt کیا گیا, partial results, possible alternatives واپس کریں
- Access failures valid empty results الگ کریں
- Transient failures subagents میں local recovery کریں؛ صرف partial non-recoverable errors کریں
results propagate کریں
- Synthesis coverage annotate میں کیا اچھی طرح supported اور gaps ہیں

5.4 Large Codebases Investigate Context Efficiently

Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص classes دینے لگتا □□
□□ کا حوالہ دیتا □□ “typical patterns” بجائے
- Scratchpad files key findings کو context boundaries میں محفوظ رکھنے □□
- Subagents کو delegate □□ verbose discovery output isolate □□ جاتی □□

- Content type مطابق render کریں: financial data tables میں، news prose میں، technical findings میں structured lists میں

Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر Claude-based Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) بنیادی ٹیکنالوجیز جن کو جانچتا ہے — یعنی وہ بنیادی علم کو جانچتا ہے۔ ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی ہیں۔

بنانا agentic systems امتحان کے سوالات حقیقی دنیا کے منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines میں شامل کرنا CI/CD کو Claude Code، ڈیزائن کرنا developer productivity tools نکالنا structured data بنانا، اور غیر ساختہ دستاویزات سے tools

هدف امیدوار

کرتا ہے آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP) — backend integration کے لیے tools اور resources
- Prompt engineering — JSON schemas، few-shot examples، data extraction templates
- Context windows — لمبی دستاویزات، multi-agent context passing
- CI/CD pipelines — automated code review، test generation
- Escalation and reliability — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (1 درست 4 میں سے)

اسکورنگ	100–1000 scale, passing score 720
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	8 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	27%
2. Tool design and MCP integration	18%
3. Claude Code configuration and workflows	20%
4. Prompt engineering and structured output	20%
5. Context management and reliability	15%

امتحانی منظر نامہ

Scenario 1: Customer Support Agent

ایک agent آپ کو Claude Agent SDK کے ساتھ returns, billing disputes, اور account issues کے ساتھ agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) کے ساتھ 80% first-contact resolution کے ساتھ 80% استعمال کرتا ہے۔

Scenario 2: Claude Code کے ساتھ Code Generation

code generation, refactoring, debugging, documentation کے ساتھ Claude Code کے ساتھ development کے ساتھ آپ کو custom slash commands اور CLAUDE.md configuration کے ساتھ integrate کے ساتھ planning mode کے ساتھ اور

Scenario 3: Multi-Agent Research System

coordinator agent specialized subagents کے ساتھ tasks: web research, document analysis, synthesis, اور report generation کے ساتھ citations کے ساتھ

Scenario 4: Developer Productivity Tools

agent engineers کے ساتھ unfamiliar codebases explore کے ساتھ boilerplate code کے ساتھ routine tasks کے ساتھ MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) کے ساتھ میں مدد دیتا ہے۔

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، pull request feedback اور pull request feedback کم سے کم false positives بننے چاہئیں کہ Prompts حاصل ہو request feedback

Scenario 6: Structured Data Extraction

کرتا ہے، validate کے ذریعے JSON schemas کو output، سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے اور درست طور پر سنبھالنے چاہئیں کہ edge cases برقرار رکھتا ہے اس high accuracy اور

Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں جن میں context window management، turns متعدد، memory strategies، execution محفوظ، tool design کے لیے instructions میں اور مہم یا متضاد، user inputs کو سنبھالنا شامل ہے

Scenario 8: Agentic AI Tools (امداد موجود نہیں — جاری مدد کریں)

میں شامل نہیں ہیں guide نہ دی لیکن ابھی تک اس candidates کی اطلاع امتحان دینے والے scenario اس میں share میں GitHub Issues کے سوالات دیکھیں، تو انہیں scenario اگر آپ نہ اصل امتحان میں اس شامل کر سکیں آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کر coverage تاکہ مہم مکمل رہے

نظریاتی بنیادیں: حصہ 1

یہ حصہ و بنیادی نظریہ بیان کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے مواد کو topic کے مطابق — اس سے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بنتی ہے

کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API میں عموماً Claude Messages API request پر کام کرتا ہے request-response model ایک شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

□□م فیلڈر:

- `model` — model selection (`claude-opus-4-6` , `claude-sonnet-4-6` , `claude-haiku-4-5`)
- `max_tokens` — response □□ زیادہ □□ زیادہ tokens
- `system` — system prompt (model کی تعریف)
- `messages` — conversation history (full history □□ بھیجنا ضروری)
- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array استعمال کرتا □□ instructional role اور ایک conversational roles دو:

- `user` — user messages, tool results (□□ `user`-role message اندر □□ `tool_result` content block الگ □□، ایک الگ `tool` role میں بھیج □□ جاتا □□،
- `assistant` — model responses (history میں □□ وقت شامل □□، بھیجنے □□ tool use requests (`tool_use` content blocks)
- `system` — top-level `system` field □□ سیٹ کیا جا سکتا □□ (□□ `turn` □□) □□ ذریعہ □□ اس نقطہ □□ آگے □□ شامل کیا جا سکتا □□ inline □□ طور پر `{"role": "system", ...}` میں `messages` □□ تابع — نیچے دیکھیں placement rules لاگو □□، مخصوص

□□ `user`-role message □□ طور پر □□ میں بھیجا جاتا □□ ایک `message` □□ `role` "tool" کو Tool results □□ شامل □□ جاتا □□ `tool_result` content block میں ایک content پر بھیج □□ جاتا □□ میں جس □□

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01...",
      "content": "..."
    }
  ]
}
```

top-level کا طور پر ظاہر ہو سکتا ہے، نہ صرف role میں بھی ایک array messages براہ راست system کیے بغیر invalidate کو cached prefix کے system field top-level کے ذریعے اس کا مقصد parameter system placement rules شامل کرنا اس کے مخصوص instructions کے درمیان conversation:

- پر ختم ہونے والے server tool use یا (توں blocks tool_result بشمول وہ جس میں) user turn یا ایک assistant turn کے فوراً بعد آنا چاہیے۔
- ہونا چاہیے element کا آخری array سے پہلے آنا چاہیے یا assistant turn کی کسی
- ملتا error کے درمیان نہیں آ سکتا — ایسا کرنے پر 400 tool_result اور اس کے block tool_use کی کسی
- پہلے والوں پر اور بعد (کے درمیان شامل کیے گئے conversation بشمول) system messages بعد میں آنے والے پر فوقیت رکھتے ہیں system field top-level کے لیے turns

محفوظ state کے درمیان Model requests بھیجیں conversation history میں مکمل API request ام بات: ہر آزاد ہوتی ہے call نہیں رکھتا؛ ہر

1.3 stop_reason

Claude API response میں stop_reason کے جو بتاتا ہے کہ:

Value	Description	Action
"end_turn"	نہ جواب مکمل کر لیا	کو نتیجہ دکھائیں user
"tool_use"	چاہتا ہے Model tool call	واپس دیں result چلائیں اور tool
"max_tokens"	ختم ہو گئی Token limit	بڑھا سکتے ہیں limit؛ response truncated
"stop_sequence"	مل گئی Stop sequence	کریں handle کے مطابق application logic

کرتے loop control ہیں — یہی signals سب سے اہم "end_turn" اور "tool_use" میں Agentic systems ہیں

1.4 System Prompt

متعین کرنا context اور behavioral rules جو instruction ایک خاص System prompt

- messages array الگ ہوتا؛ الگ system field کا حصہ نہیں ہوتا؛
- user messages پر فوقیت رکھتا ہے
- ہو کر پوری گفتگو میں لاگو رہتا ہے load ایک بار
- کی وضاحت کے لیے استعمال ہوتا ہے output format، اور role، constraints،

کو غیر ارادی طور پر متاثر کر سکتی ہے مثال tool selection wording کی system prompt: امتحان کے لیے اہم ضرورت سے زیادہ get_customer کو model کی تصدیق کریں "جیسی ہدایت customer کے طور پر" ہمیشہ استعمال کرنے پر لا جا سکتی ہے


```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description نکات:

- Minimal descriptions کا سبب بنتی selection غلط mechanism کا بنیادی tool selection کی Description ہیں
- edge cases، کیا input format، کیا کرتا، کیا واپس کرتا tool: میں یہ شامل کریں Description، اور یہ کب استعمال ہو
- Overlapping descriptions سے بچیں
- Built-in tools اور MCP tools overlap کو تو واضح بنائیں MCP tool

2.3 tool_choice

Value	Behavior	کب استعمال کریں
<code>{ "type": "auto" }</code>	خود طے کرتا Model	default عام
<code>{ "type": "any" }</code>	کرنی tool call کو لازماً Model	structured output جب چاہیے
<code>{ "type": "tool", "name": "extract_metadata" }</code>	وگي call لازماً tool خاص	forced first step / order کی

2.4 JSON Schemas for Structured Output

reliable لینے کا سبب structured output سے Claude استعمال کرنا `tool_use` کی سائنہ JSON schemas کی semantic correctness نافذ کرتا، مگر required structure کم کرتا اور syntax errors طریقہ سے ضمانت نہیں دیتا

Design اصول:

- رکھیں جو ہمیشہ ملنے چاہئیں fields required صرف وہ
- استعمال کریں nullable fields کی missing data ممکنہ
- enum values جیسے "unclear" اور "other" information شامل کریں تاکہ

2.5 Syntax اور Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، field غلط	JSON schema کے ساتھ tool_use
Semantic	Totals match نہیں کرتے، value field غلط	Validation checks، feedback retry

3 بنانا Claude Agent SDK — Agentic Systems

دستاویزات: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

3.1 Agentic Loop

Agentic loop میں Claude کے actions کی sequence چلاتا ہے، بلکہ Claude میں صرف جواب نہیں دیتا، بلکہ Claude میں actions کی sequence چلاتا ہے:

- Claude کو tools کے ساتھ request بھیجیں
- Response لیں
- stop_reason: چیک کریں:
 - "tool_use" → tool چلائیں، result history کے ساتھ دوبارہ request میں ڈالیں، پھر دوبارہ
 - "end_turn" → task مکمل، نتیجہ کو دیں
- مکمل ہونے تک دہرائیں

Text parsing یا arbitrary iteration دیکھیں کہ `stop_reason == "end_turn"` کے لیے completion signal: درست قابلِ اعتماد نہیں ہے

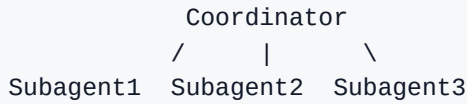
3.2 AgentDefinition

```
agent = AgentDefinition(  
    name="customer_support",  
    description="Handles customer requests for returns and order issues",  
    system_prompt="You are a customer support agent...",  
    allowed_tools=["get_customer", "lookup_order", "process_refund",  
    "escalate_to_human"],  
)
```

اصول اپنائیں: Least privilege کے allowed_tools اور، name، description، system_prompt: مہم چیزیں

3.3 Coordinator اور Subagents

Multi-agent systems میں hub-and-spoke topology اکثر استعمال کرتی ہے:



Coordinator task کو توڑتا ہے، subagents کے نتائج جوڑتا ہے، کام سونپتا ہے، منتخب کرتا ہے، errors handle کرتا ہے، جواب پہنچاتا ہے final تک user کو، اور

نہیں کرتا۔ inherit خود بخود parent history الگ ہوتا ہے؛ وہ context کا subagents کا ماحول:

3.4 Task Tool

Subagents Task tool کے ذریعے spawn کیے جاتے ہیں۔ Coordinator کے allowed_tools میں "Task" ہونا چاہیے۔

context passing:

- Task: "Analyze the document" کافی نہیں
- Task: "Analyze this document. Full text: ... Output schema: ..." درست ہے

Coordinator ایک response میں multiple Task s لے سکتا ہے، لہذا parallel subagents بھیج سکتا ہے،

3.5 Hooks

Hooks lifecycle کے مخصوص points پر work کیے جاتے ہیں۔

PostToolUse tool result کو model پہلے سے normalize تک پہنچاتا ہے۔

```

@hook("PostToolUse")
def normalize_dates(tool_result):
    return normalized_result

```

PreToolUse policy violation block کر سکتا ہے:

```

@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)

```

Critical business rules ہوتی ہیں۔ prompt instructions probabilistic ہوتے ہیں؛ hooks deterministic: یاد رکھیں۔

باب 4: Model Context Protocol (MCP)

4.1 MCP کیا ہے

MCP resource سے جوڑنا اس کے تین بنیادی Claude کو external systems جو open protocol ایک types ہیں:

1. **Tools** — actions کے functions
2. **Resources** — context data (documentation, schemas, catalogs)
3. **Prompts** — predefined task templates

4.2 MCP Servers

MCP server ایک process جو tools/resources expose کرتا ہے Connect پر tools discover ہوتا ہے۔ مطابقت استعمال ہوتی ہے۔ descriptions ہیں اور

4.3 Configuration

Project config (`.mcp.json`) کے لیے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

User config سے آتی ہے۔ secrets environment variables میں ہوتی ہے؛ Project config version control (`~/.claude.json`) personal/experimental servers کے لیے ہوتی ہے۔

4.4 isError

MCP tool errors میں `isError: true` Generic errors کے بجائے structured error کو واپس کریں:

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable."
  }
}
```

4.5 Resources

Resources □ and data □ agent □ action □ □ پڑھ سکتا □: schemas, docs, content catalogs, summaries □ □ exploratory tool calls □ □ کم کرتا □

اور Claude Code — Configuration: باب 5 Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

5.1 CLAUDE.md Hierarchy

تین سطحوں پر سو سکتا Claude.md

- **User-level:** `~/ .claude/CLAUDE.md` — پر لاگو user صرف اسی
- **Project-level:** `.claude/CLAUDE.md` یا `root` `CLAUDE.md` — team کے لیے پوری
- **Directory-level:** subdirectories میں `CLAUDE.md` — folder conventions مخصوص

5.2 @path Syntax

و: config modular کیا جا سکتا ہے تاکہ external files refer @path میں CLAUDE.md

Coding standards @./standards/coding-style.md میں ہیں
Test rules @./standards/testing-requirements.md میں ہیں

5.3 .claude/rules/

conditional loading کے ذریعے paths کرتا ہے اور organize کے لحاظ سے topic کو rules/claude/rules/ دیتا ہے:

```
---
paths: ["src/api/**/*"]
---
```

استعمال کریں۔ explicit error handling اور async/await کے لیے API files

5.4 Commands اور Skills

`.claude/commands/` project commands کے لیے، جبکہ `~/claude/commands/` personal commands کے لیے۔
ملتی ہیں۔ skills کے ساتھ frontmatter میں `.claude/skills/` کے لیے۔

5.5 Skills اور context: fork

context: fork skill کو isolated context میں چلانا، allowed-tools سے tool access اور محدود کریں،
argument-hint سے required input مانگیں۔

5.6 Planning Mode

Planning mode کے لیے، بہترین unfamiliar codebases اور multiple approaches، تبدیلیوں architectural بڑی
کے لیے، بہتر changes کے واضح Direct execution اور

5.7 /compact اور /memory

/compact سے context compress کرتا ہے، اور /memory سے notes اور preferences میں مدد دیتا ہے۔

5.8 Claude Code CLI for CI/CD

کے لیے استعمال ہوتا ہے: `-p / --print` non-interactive mode

```
claude -p "Analyze this pull request for security issues"
```

Structured output کے لیے `--output-format json` اور `--json-schema` استعمال کریں۔

باب 6: Prompt Engineering — Advanced Techniques

دستاویزات: [Prompt Engineering](#) | [Anthropic Cookbook](#)

6.1 Few-shot Prompting

Few-shot prompting میں 2–4 examples کے expected behavior دکھایا جاتا ہے،
vague instructions سے example ambiguity زیادہ مؤثر ہے کیونکہ کم کرتا ہے۔

6.2 Explicit Criteria

decide severity کرنا، اور ignore کرنا، کیا flag دیں: کیا clear criteria بجائے vague instructions کرنا

6.3 Prompt Chaining

Complex task کو focused steps میں توڑیں: local analysis، integration pass، attention dilution کم کرتا

6.4 Interview Pattern

Claude domain سامنے لاتا، خاص طور پر جب design ambiguity پوچھنا clarifying questions سے unfamiliar

6.5 Validation and Retry

Extraction fail تو original document، incorrect output، اور specific validation error ساتھ retry کرے

6.6 Self-correction

Model stated value اور calculated value؛ conflict دونوں نکال؛ discrepancy handle کرے

7: Message Batches API

Message Batches: دستاویزات

7.1 Overview

Message Batches API asynchronous processing 50% savings، up to 24 hours window، اور multi-turn tool calling supported؛ custom_id requests اور responses link

7.2 کب استعمال کریں

Pre-merge checks اور interactive tasks؛ synchronous API، overnight reports اور bulk jobs؛ Batch API استعمال کریں

7.3 custom_id

custom_id result کو original document سے جوڑنا، failures identify اور صرف failed items کو دوبارہ submit کرنے میں مدد دیتا ہے

8: Task Decomposition Strategies

8.1 Fixed Pipelines

Predictable tasks کے لیے fixed pipeline جیسے Document → Extraction → Validation → Final output

8.2 Dynamic Decomposition

Open-ended tasks میں subtasks intermediate results کی بنیاد پر بنتے ہیں

8.3 Multi-pass Code Review

Large PRs کے لیے per-file analysis کے بعد cross-file integration pass تاکہ attention dilution کم ہو

9: Escalation اور Human-in-the-Loop

9.1 Escalate کب کریں

Explicit user request, policy gaps, progress نہ ہونا، threshold سے اوپر financial action، یا ambiguous matching کے لیے Sentiment analysis اور self-rated confidence unreliable کی صورت میں escalate کریں

9.2 Escalation Patterns

human کو hand off یا explicit insistence یا policy gap کے لیے کی کوشش کریں، پھر اگر resolve نہ ہو، مسئلہ کریں

9.3 Structured Handoff

Escalation پر human شامل کریں تاکہ customer ID, issue summary, actions taken, اور recommended action کے مکمل context دیا جاسکے

9.4 Confidence Calibration

Field-level confidence scores, calibration on labeled data, اور stratified sampling استعمال کریں

10 Error Handling میں Multi-agent Systems: باب 10

10.1 Error Categories

Transient errors retryable ہیں، validation errors input fix ہیں مانگتی، business errors policy-driven ہوتی ہیں کی جاتی ہیں۔ permission errors generally escalate ہیں، اور

10.2 Anti-patterns

Generic “search unavailable” response, silent suppression, workflow abort یا پورا کرنا نقصان دہ۔ Structured errors واپس کریں

10.3 Structured Subagent Error

Structured error میں failure type, attempted query, partial results, alternative approaches, اور coverage شامل کریں impact

10.4 Final Synthesis میں Coverage

Report میں کون سی sections fully covered ہیں، کون سی partially covered ہیں، اور کون سی gaps باقی ہیں

11 Context میں Production Systems: باب 11

11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block میں رکھیں تاکہ summarization باوجود critical data کے ہو سکے

11.2 Tool Results Trimming

PostToolUse hook سے relevant fields رکھیں

11.3 Position-aware Input

کم □□ lost-in-the-middle effect آخر میں رکھیں تاکہ □□ action items شروع میں اور Important findings

11.4 Scratchpad Files

محفوظ □□□□ context میں long investigations میں لکھیں تاکہ □□ scratchpad findings verbose

11.5 Subagents کو Delegate کرنا

رکھیں □□ summary میں صرف main context میں دیں اور exploration subagents

11.6 Structured State Persistence

ممکن □□ crash recovery کر □□ تاکہ □□ export میں JSON file یا state manifest اپنی agent □□

12 کو محفوظ رکھنا Provenance: باب 12

12.1 Attribution Loss

رکھیں □□ confidence اور ، publication date ، source URL □□ ساتھ Claim

12.2 Conflicting Data

کریں □□ conflict flag □□ ساتھ محفوظ رکھیں اور attribution دیں تو دونوں کو values مختلف sources اگر

12.3 Dates

دیکھیں □□ publication dates سمجھنے □□ سد □□ □□ contradiction کو Temporal differences

12.4 Content Type □□ Render

time series میں ، اور technical findings structured lists ، news prose میں ، میں financial data tables chronological order □□ دکھائیں

باب 13: Claude Code Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
تلاش کرنا file کا pattern	Glob	<code>**/*.test.tsx</code>
file contents میں search	Grep	function name, error message
پڑھنا file	Read	analysis کا لپکا
بنانا file نیا	Write	scratch کا
precise edit	Edit	unique text match
shell command	Bash	git, npm, tests

13.2 Incremental Investigation

دیکھیں کا consumer files، پھر usages، پھر entry points، پڑھیں؛ پلکا files ایک ساتھ سب

13.3 Read + Write Fallback

کریں کا Write کریں، پھر programmatically کریں، تبدیلی Read کا و تو Edit fail اگر

امتحانی ڈومین نوٹس: II حصہ

1 ڈومین: Agent Architecture اور Orchestration (27%)

Agentic loops, coordinator–subagent architecture, Task کا ذریعہ spawning, hooks, escalation, اور multi-step workflows پر فوکس کریں

2 ڈومین: Tool Design اور MCP Integration (18%)

Tool descriptions, tool_choice, JSON schema output, structured errors, MCP servers, resources, اور built-in tools کا فرق کو سمجھیں

- B) Model کی context window exceed ہو گئی
- C) Claude API کو session_id parameter درکار
- D) شامل نہیں کر رہی [درست] messages array messages میں پچھلے application آپ کی

D کیوں: Claude پاس server-side memory کے request stateless ہیں — API call کے پاس Claude turns کے پاس پچھلے conversation history میں مکمل messages array کا کوئی علم نہیں؛ دو Claude architecture (C) session_id اور (A) Vector databases میں؛ context window overflow (B) ناممکن ہے exchange کے messages

سوال 65

History تک پہنچ گئی 78,000 tokens conversation، بعد cooking session صورتحال: 40 منٹ کے شامل ہیں آپ کو اہم معلومات discussion اور عمومی allergies, recipe scaling, clarified cooking terms، کم کرنے کے tokens محفوظ رکھنے کے

کدام درمیان بہترین توازن رکھتا ہے؟ token reduction اور approach preservation کونسا

- A) خلاصہ کریں conversation history پوری
- B) رکھیں tokens صرف سب سے حالیہ 20,000
- C) خلاصہ کریں، اور discussion نکالیں، عمومی structured data (allergies, quantities, preferences) م رکھیں [درست] verbatim کو exchanges حالیہ
- D) retrieve کے ذریعہ متعلق حصہ semantic search کریں اور conversation externally store مکمل

C کیوں: Hybrid approach کم cost سب سے زیادہ valuable پر سب سے زیادہ cost کم summarization (summarization) ہیں extract میں compact structured block ایک critical facts جیسے recipe quantities اور exchanges ہوتی ہیں، اور حالیہ discussion summarize عمومی، (سب سے بچاؤ کے precision loss کے دوران conversational coherence کے dietary information م B اور A Options میں verbatim کے لیے conversational coherence کے لیے ضرورت سے زیادہ پیچیدہ کوئی cooking session ایک D ہیں؛

سوال 66

topics اور preferences کا خلاصہ assistant میں conversations رپورٹ کرتے ہیں کے طویل Users صورتحال رکھتی ہے message pairs صرف آخری 25 implementation حساب کھو دیتا ہے آپ کی موجود

کیا ہے؟ solution سب سے مؤثر

- A) Hybrid approach: messages verbatim [درست] خلاصہ کرتے ہوئے حالیہ messages پرانے
- B) vector similarity search پر conversation history مکمل
- C) Window 50 کے message pairs تک بڑھائیں
- D) آگے جوڑیں running summary خلاصہ کریں اور dropped messages میں turn

A کیوں: Hybrid approach exact recent context کے دونوں پہلوؤں کو حل کرتی ہے (conversational coherence کے لیے) م compressed representation کی preferences جبکہ خلاصہ کی (کے لیے) م miss کو context م (B) Vector search صرف اسی مسئلہ کو ملتوی کرتا ہے (C) بڑھانا Window رکھتی ہے semantically سے current query کر سکتی ہے جو

سوال 67

بڑھ جاتی latency سے زیادہ ٹوکن ہیں تو 50 turns conversations رپورٹ کرتے ہیں کہ جب Users: صورتحال بھی costs ہیں اور

بنیادی وجہ کیا ہے؟

- A) شامل ہوتی ہیں [درست] conversation history کے ساتھ پوری API request
- B) Model gradually responses generate کرتا ہے
- C) History کے ساتھ database operations میں جاتے ہیں بڑھنے کے ساتھ
- D) بنانا ہے internal user profile کی ضرورت والا processing زیادہ Model

A میں پوری array messages کو request — کے stateless مکمل طور پر API کی Claude: کیوں A زیادہ request بڑھتی ہیں، کے conversations شامل کرنی ہوتی ہیں جیسے جیسے conversation history کے درمیان کوئی Model calls دونوں بڑھانا cost اور directly processing latency لے جاتی ہے، جو tokens internal state رکھتا ہے (D غلط ہے)

سوال 68

تک بڑھ گئی ہیں جب conversation history 85,000 tokens کے بعد weekly sessions: صورتحال: تین مہینوں کے discussions پچھلی assistant کے بارے میں کیا نتیجہ اخذ کیا تھا؟، تو theme کے isolation پوچھتا ہے "م نہ user کرنے کے بجائے عام جوابات دیتا ہے reference

کیا ہے؟ approach سب سے مؤثر

- A) Rolling window truncation
- B) Key conclusions capture کے progressive summarization
- C) Relevant exchanges retrieve کے لیے [درست] semantic embeddings
- D) Discussion conclusions mark کے لیے structured XML tags

C scale کو discussion کے جو تین مہینوں کی approach واحد semantic search پر Conversation history Rolling window truncation کر سکتی ہے specific relevant exchanges surface پر demand کر سکتی ہے جبکہ truncation (A) history کے Progressive summarization (B) discussions کا بیشتر حصہ ضائع کر دے گی history (A) truncation پوچھتا ہے ہیں users کہو دیتی ہے جن کے بارے میں specific conclusions، کرتی ہے compress میں abstractions

سوال 69

کرتا ہے، system prompt guidelines follow میں 10–15 Claude، کے دوران QA testing: صورتحال کے اندر token limits ابھی Conversation آ جانی ہے deviation میں responses لیکن بعد کے

کیا ہے؟ solution بہترین

- A) Behavioral guidelines کے user message کو پلے میں منتقل کریں
- B) 20 turns کے بعد نئی conversation شروع کریں
- C) Conversation breakpoints پر guidelines reinforce والے user-role messages insert [درست] کریں
- D) Non-compliant responses کو regenerate کے لیے post-response validation کرنے کے

history کا براہ راست مقابلہ کرتا ہے، periodic injection instruction drift کا Behavioral reminders کیوں C user کو پہلی Guidelines کر کے re-establish constraints پر regular intervals کو ساتھ accumulate message نہ، Post-response validation (D) corrective کم کرتا authority ان کا (A) میں منتقل کرنا preventive اور significant latency شامل کرتی ہے

سوال 70

adaptation rules اور teaching methodology کو جو 2,800-token system prompt میں AI tutor 2,800 صورتحال: آپ کے کو نظر انداز کرنا شروع کر دیتا ہے، assistant proficiency levels بعد 12 turns کرنا

کیا؟ fix سب سے مؤثر

- A) 5-4 turns میں inject reminders کریں
- B) Verbose rules کو proficiency-level adaptation demonstrate والے few-shot examples سے replace کریں [درست]
- C) Critical rules کو system prompt میں رکھیں
- D) Responses evaluate کر کے اگر difficulty level match کریں اور regenerate نہ کرے تو

abstract rules کیونکہ vulnerable کے لیے 2,800-token system prompt drift Declarative rules کیوں B سے concrete few-shot examples کو Verbose rules کا تقاضا کرتے ہیں، reasoning سے model میں turn clear کرنے کے لیے match کو model، کریں، correct proficiency-level adaptation demonstrate کرنا جو replace رہتا ہے، reliably follow سے زیادہ abstract instructions میں turns دیتا ہے — یہ کئی behavioral patterns

سوال 71

clarifying کرنا، اور reasoning explain برقرار رکھنا، اپنی enthusiastic tone کو assistant صورتحال: آپ کے کو behavioral guidelines ان کیوں؟ define

کیوں؟ behavioral guidelines ان کیوں؟ define

- A) user message سے پہلے prepend کریں
- B) System prompt میں [درست]
- C) assistant message سے پہلے
- D) Environment variables میں

persistent behavioral کو والی apply میں conversation خاص طور پر پوری System prompt کیوں B redundant (A) سے پہلے شامل کرنا user message کے لیے ڈیڑھ گھنٹہ کے لیے guidelines اور constraints assistant message (C) unreliable کیونکہ model کے statements اپنے پہلے deviate overhead سے Environment variables (D) model کے behavior کو Environment variables کر سکتا ہے

سوال 72

جیسے "یقیناً!" اور "میں مدد کرنے میں خوش ہوں!" رپورٹ کرتے ہیں: Users repetitive response openings صورتحال

کیا approach سب سے مؤثر

- A) Direct response opening ساتھ partial assistant message append [درست] کریں
- B) Temperature setting کم کریں
- C) Greetings کو post-process کرنا، responses کو لے کر
- D) System prompt کی phrases میں ان instructions سے بچنا شامل کریں

A کیوں: Assistant کو direct response کے generation level پر greeting patterns کو generate کرنے کے لیے opening phrases کو بھرنے کے لیے model prefill سے روکتا ہے۔ Post-processing (C) میں reliable مدد کر سکتی ہے لیکن کم (D) System prompt instructions، specific phrase patterns کرتا ہے، (B) randomness control Temperature کو workaround

سوال 73

user کو گنا دینا جبکہ package ship user کا کرتا ہے کہ notify کو system آپ کے webhook صورتحال: ایک شامل کرنا naturally میں response اس کے اگلا assistant کر رہا ہے آپ چاہتے ہیں کہ actively chat

کیا approach بہترین

- A) Shipping status کو system prompt میں شامل کریں
- B) synthetic user message فوری بھیجیں
- C) کرنا پر مجبور کریں کہ status tool call کو assistant میں turn کر
- D) Status update کو اگلا user message کے prefix طور پر append [درست] کریں

D کیوں: Status update کو aگلا user message کے prefix طور پر شامل کرنا کہ flow کو interrupt بغیر کہ natural conversation boundary پر real-time context inject کرنا System prompt modify کرنا (A) session rebuild کر سکتا ہے کہ Synthetic user message (B) natural dialogue flow کی ضرورت کو (C) کرنا tool call force میں events rare جب wasteful اس وقت (C) کرنا tool call force

سوال 74

Assistant 4+ clarifying questions میں requests کریں "جیسے venue book اکثر پارٹی کے لیے Users صورتحال questions 35 ہوتا ہے abandonment % پوچھتا ہے، جس سے 35

کرتا ہے؟ improve کو بہترین طریقہ سے approach trade-off کونسا

- A) Hidden defaults ساتھ آگے بڑھیں
- B) میں پوچھیں کہ compound message ایک clarifying questions تمام
- C) Assumptions explicitly بیان کریں اور corrections [درست] آگے بڑھیں
- D) Structured intake form استعمال کریں

C کیوں: Assumptions explicitly بیان کرنا اور آگے بڑھنا کہ user کو غلط response کو فوری، مفید user بیان کرنا اور آگے بڑھنا کہ Hidden defaults (A) user کو بتانا کہ کیا user بھر بھی Compound question list (B) upfront effort سے friction کم (D) form کی بجائے زیادہ شامل کرتا ہے

سوال 75

turns rules follow استعمال کرتا ہے ابتدائی assistant contractor-persona system prompt صورتحال: آپ کا Conversation length 2,500 tokens صرف دیتا ہے assistant generic advice تک 7 turn کرتا ہے، لیکن

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) System prompts initial behavior establish کرتے ہیں صرف
- B) Turns accumulate کمزور ہوتی ہے attention کی model ہونے کے ساتھ
- C) Accumulated assistant responses system prompt اثر کو dilute [درست] کرتے ہیں
- D) System prompt صرف ایک بار بھیجا جاتا ہے

system prompt کے جمع ہونے میں assistant responses میں conversation history کیوں جیسے جیسے C کے assistant-generated content بڑھنے والے proportion کا text کرنے والے reflect کو behavioral constraints کے بجائے اپنے پیمائش system prompt instructions پر بھی token lengths چھوٹے Model کم ہوتا جاتا ہے relative کے compound کو drift، کرتا ہے follow کو زیادہ patterns کے outputs

سوال 76

کئی Assistant "میں مدد کر سکتے ہیں؟" report کرتے ہیں جیسے "کیا آپ requests میں Users صورتحال abandonment % جس سے 40، (کیا ہے deadline؟ کیا مدد؟ report کون سی) کرتا ہے respond سوالات پوچھ کر ہوتا ہے

کیا ہے solution بہترین

- A) Reasonable assumptions انہیں اور explicitly کریں،
- B) Respond smaller model کرنے سے پیمائش ambiguity classify کریں کے ساتھ
- C) Assumptions predefined interpretations بیان کیے بغیر استعمال کریں
- D) Assistant turn کو رکنے تک محدود کریں clarifying question میں ایک

رکھتے ہیں in control اور informed کو user کے ساتھ آگے بڑھنا Reasonable stated assumptions کیوں A کرتی ہے confuse کو (C) users Silent predefined interpretations کو مکمل طور پر ختم کرتا ہے back-and-forth کی back-and-forth turns بھی (D) One-question limit، match ان کے ارادے سے response میں جب infrastructure اور latency مسئلہ حل کیے بغیر (B) core UX Smaller classification model ضرورت ہے شامل کرتا ہے complexity

عملی مشقیں

Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration، structured errors، اور escalation کے ساتھ agent loop بنائیں

Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md, custom commands, path-specific rules, اور MCP servers configure کریں

Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema, validation/retry loops, اور Message Batches API کے ساتھ extraction pipeline بنائیں

Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration, explicit context passing, error propagation, اور source tracking implement کریں

Appendix: Technologies and Concepts

Technology	Key aspects
Claude Agent SDK	agent loops, <code>stop_reason</code> , hooks, <code>Task</code> , <code>allowedTools</code>
MCP	servers, tools, resources, <code>isError</code> , <code>.mcp.json</code>
Claude Code	CLAUDE.md, <code>.claude/rules/</code> , <code>.claude/commands/</code> , <code>.claude/skills/</code> , planning mode
Claude Code CLI	<code>-p</code> , <code>--output-format json</code> , <code>--json-schema</code>
Claude API	<code>tool_use</code> , <code>tool_choice</code> , <code>stop_reason</code> , <code>max_tokens</code>
Message Batches API	50% savings, 24-hour window, <code>custom_id</code>
JSON Schema	required/optional, nullable, enum values
Few-shot prompting	ambiguous scenarios کے لیے examples
Prompt chaining	sequential decomposition
Context window	token budget, lost-in-the-middle
Confidence calibration	field-level scoring, stratified sampling

Out-of-Scope Topics

یہ موضوعات امتحان میں شامل نہیں ہیں:

- Claude models کی fine-tuning
- Authentication, billing, یا account management
- implementation کی تفصیلی programming languages مخصوص
- MCP servers کی hosting/infrastructure
- Claude کی internal architecture یا model weights
- Constitutional AI, RLHF, یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

Preparation Recommendations

1. بنائیں agent کا ساتھ ایک Claude Agent SDK
2. کریں configure پر real project کو Claude Code
3. کریں test اور MCP tools design
4. بنائیں Data extraction pipeline
5. کریں Prompt engineering practice
6. سیکھیں Context management patterns
7. سمجھیں human-in-the-loop workflows اور Escalation
8. حل کریں Practice exam